

Aprendizaje Continuo y Adaptación en Tiempo Real para Robots Móviles en Entornos Bancarios: Un Enfoque basado en SAC con Mitigación del Olvido Catastrófico



**Universidad
Internacional
de Valencia**

Titulación: Maestría
inteligencia artificial.

Curso académico

2025 – 2026

en Alumno/a:

Granda Rodriguez Jenniffer
Lisette

D.N.I: 0925330805

Convocatoria: Tercera

De:

 Planeta Formación y Universidades

Índice

Resumen	7
1. Introducción	8
2. Planteamiento del problema	10
3. Objetivos.....	12
3.1. Objetivo Principal	12
3.2. Objetivos Específicos	12
3.3. Alcance	12
4. Estado del Arte y Marco teórico	14
4.1. Estado del arte	14
4.2. Marco Teórico	19
4.2.1. Introducción y fundamentos conceptuales	19
4.2.2 Fundamentos de la Robótica Móvil Autónoma	19
4.2.3. Aprendizaje Supervisado, No Supervisado y por Refuerzo	21
4.2.4. Aprendizaje por Refuerzo Profundo para Navegación.....	22
4.2.6. Gazebo como simulador para aprendizaje por refuerzo	29
4.2.7. TurtleBot3 Waffle Pi: plataforma robótica de referencia	30
4.2.8. Aprendizaje en Línea de Final Abierto	31
4.2.9. Métricas de Evaluación en Navegación Robótica	32
5. Metodología.....	34
5.1. Etapas 1: Diseño del entorno de simulación.....	35
5.1.1. Representación del Estado	35
5.1.2. Espacio de Acciones.....	36
5.1.3. Entorno Simulado y Puntos de Interés.....	37
5.2. Etapas 2: Implementación del Agente	38
5.2.1. Política de Navegación: Soft Actor-Critic	39
5.2.2. Modelo de Movimiento Incremental	40
5.2.3. Función de Recompensa Multi-objetivo	41
5.3. Etapas 3: Integración y Entrenamiento	42
5.3.1. Curriculum Progresivo	44
5.3.2. Factor de Descuento Adaptativo	45
5.2.3. Hiperparámetros del Sistema.....	46
5.4. Etapas 4: Evaluación y Análisis.....	48



5.4.1.	Metricas de evaluación	48
5.4.2.	Métricas de Entrenamiento	49
6.	Resultados.....	50
6.1.	Recompensa acumulada	50
6.2.	Comportamiento de evasión de obstáculos	50
6.3.	Eficiencia de trayectoria y exploración del entorno	51
6.4.	Distribución de métricas clave	52
7.	Conclusión y trabajos futuros.....	53
7.1.	Conclusiones.....	53
7.2.	Trabajos futuros	54
8.	Referencias	56
	Apéndice I.....	61
	A.1. Arquitectura detallada del agente SAC.....	61
	A.2. Función de recompensa completa	61
	A.3. Ecuaciones de actualización SAC.....	62
	Anexos I.....	64
	Anexo A. Especificaciones técnicas del TurtleBot3 Waffle Pi	64
	Anexo B. Configuración del entorno de simulación.....	64
	Anexo C. Entorno bancario simulado	65
	Anexo D. Repositorio de código	65

Índice de ilustraciones

Figura 1. Problema típico en la planificación de trayectorias de robots. Tomado de ...	17
Figura 2. Arquitectura modular de percepción, planificación y control en robots móviles autónomos. Adaptado de Siegwart et al. (2011).	20
Figura 3. Ciclo de interacción agente–entorno en el Proceso de Decisión de Markov. Tomado de Sutton & Barto (2018).	22
Figura 4. Arquitectura del algoritmo Soft Actor-Critic. Tomado de Haraguchi, K. et al. (2023).	23
Figura 5. Componentes de una función de recompensa densa para navegación de robots móviles. Tomado de Botteghi et al. (2020).	26
Figura 6. Esquema de curriculum learning adaptativo con realimentación de rendimiento por tarea. Tomado de Narvekar et al. (2020).	28
Figura 7. Degradación del rendimiento por olvido catastrófico y su mitigación mediante EWC. Tomado de Kirkpatrick et al. (2017).	29
Figura 8. Entorno bancario simulado en Gazebo.	37
Figura 9. ATM 1 y ATM 2.	38
Figura 10. Ventanillas.	38
Figura 11. Servicio al cliente.	38
Figura 12. Recompensa acumulada por episodio de evaluación (N = 100). La línea discontinua indica la media (194.2).	50
Figura 13. Colisiones por episodio de evaluación. El 84% de los episodios registró 0 colisiones.	51
Figura 14. Eficiencia de trayectoria por episodio.	52
Figura 15. Distribución de métricas clave (N = 100): goals completados, colisiones, eficiencia de trayectoria y cobertura del entorno.	52

Índice de tablas

Tabla 1. Etapas del desarrollo metodológico.....	28
Tabla 2. Composición del vector de estado (370 dimensiones).....	29
Tabla 3. Puntos de interés (POIs) del entorno bancario.....	31
Tabla 4. Arquitectura de redes neuronales del agente SAC.....	32
Tabla 5. Configuración del MotionModel con EWC.....	34
Tabla 6. Componentes de la función de recompensa.....	35
Tabla 7. Hiperparámetros del Actor y Críticos (SAC).....	37
Tabla 8. Hiperparámetros del MotionModel.....	37
Tabla 9. Hiperparámetros de Curriculum y Control.....	38
Tabla 10. Métricas de evaluación del sistema.....	39
Tabla 11. Métricas de entrenamiento y diagnóstico.....	39

Resumen

La navegación autónoma de robots móviles en entornos interiores dinámicos representa un desafío fundamental en robótica, donde los enfoques clásicos basados en mapas estáticos resultan insuficientes ante la presencia de obstáculos móviles y cambios ambientales. Este trabajo presenta un sistema de navegación basado en aprendizaje por refuerzo profundo utilizando el algoritmo Soft Actor-Critic (SAC), diseñado para operar en un entorno bancario simulado con personas en movimiento. La arquitectura propuesta integra tres componentes principales: (1) un agente SAC con redes Actor-Critic de tres capas ocultas (256 neuronas cada una) que procesa un espacio de observación de 370 dimensiones incluyendo lecturas LiDAR, información propioceptiva y variables de progreso de tarea; (2) un MotionModel predictivo con consolidación mediante Elastic Weight Consolidation (EWC) para mitigar el olvido catastrófico durante transiciones de fase; y (3) un curriculum progresivo de cuatro fases que incrementa gradualmente la complejidad de la tarea desde un único punto de interés hasta secuencias completas de cuatro destinos. El sistema fue entrenado y evaluado en Gazebo con ROS 2, utilizando un TurtleBot3 Waffle Pi en un entorno bancario de 400 m² con 10 personas dinámicas. Los resultados experimentales sobre 100 episodios de evaluación demuestran una tasa de éxito del 84% en navegación libre de colisiones, una recompensa acumulada media de 194.2 puntos, y una eficiencia de trayectoria del 72.3%. El análisis de las métricas revela que el agente desarrolló comportamientos emergentes de evasión anticipada, manteniendo distancias de seguridad superiores a 0.35 m en el 91% de los pasos. Estos hallazgos validan la efectividad del enfoque propuesto para navegación reactiva en entornos con incertidumbre dinámica, estableciendo una base para futuras extensiones hacia transferencia sim-to-real y aprendizaje continuo en línea.

Palabras clave: aprendizaje por refuerzo profundo, navegación autónoma, Soft Actor-Critic, robots móviles, olvido catastrófico, curriculum learning, ROS 2, Gazebo.

1. Introducción

La exploración robótica autónoma en entornos interiores desconocidos constituye uno de los problemas fundamentales de la robótica móvil, al integrar percepción, planificación y control en un mismo ciclo operativo (Thrun et al., 2005). Esta problemática cobra especial relevancia cuando se trata de un robot de servicio móvil que debe operar en el interior de agencias bancarias, donde no solo se requiere eficiencia en la cobertura del espacio, sino también seguridad, fluidez en la navegación y capacidad de adaptación ante la presencia constante de personas.

Una sucursal bancaria es un entorno estructurado pero dinámico. Existen zonas de circulación intensa, áreas de espera, módulos de atención, cajeros automáticos y oficinas internas, cada una con características espaciales distintas. Además, la disposición del mobiliario puede variar entre sucursales o modificarse con el tiempo. Para un robot de servicio, esto implica enfrentarse a incertidumbre en el modelo del entorno, variaciones en la interacción con el suelo y perturbaciones generadas por el movimiento de clientes y empleados. En consecuencia, la exploración no es únicamente un problema geométrico, sino también dinámico y adaptativo.

En los últimos años, el aprendizaje por refuerzo profundo (Deep Reinforcement Learning, DRL) ha emergido como un paradigma prometedor para la toma de decisiones secuenciales en dominios complejos. Los trabajos pioneros de Mnih et al. (2015) demostraron que es posible aprender políticas de control directamente a partir de datos sensoriales en bruto, mientras que algoritmos posteriores como Soft Actor-Critic (SAC) (Haarnoja et al., 2018) introdujeron mejoras significativas en estabilidad y eficiencia de muestreo mediante la maximización conjunta de recompensa y entropía. En el ámbito de la navegación robótica, estos enfoques han permitido desarrollar sistemas capaces de operar en entornos parcialmente observables sin requerir un mapa previo del espacio (Tai et al., 2017; Zhu et al., 2017).

Sin embargo, los métodos basados en DRL presentan limitaciones prácticas que deben considerarse. En particular, suelen requerir grandes volúmenes de interacciones, tiempos de entrenamiento prolongados y recursos computacionales significativos (Henderson et al., 2018). En el contexto de un robot de servicio que opera dentro de una agencia bancaria, donde se espera respuesta en tiempo real y se dispone de hardware embebido con capacidad limitada, estas exigencias pueden resultar poco prácticas si no se emplean estrategias de entrenamiento eficientes y entornos de simulación adecuados.

Sin embargo, los métodos tradicionales de exploración, como la exploración basada en fronteras (Yamauchi, 1997) y los planificadores basados en muestreo aleatorio como el RRT (LaValle, 1998), han demostrado ser herramientas sólidas y ampliamente validadas en robótica móvil, gracias a sus fundamentos matemáticos claros y su eficiencia computacional. No obstante, estos enfoques asumen modelos de movimiento idealizados que no siempre reflejan las condiciones reales de operación, donde pueden presentarse deslizamientos, desviaciones o errores acumulados de odometría. En entornos como una sucursal bancaria, que exigen trayectorias precisas y suaves, estas discrepancias pueden afectar la eficiencia y naturalidad de la navegación.

Por estas razones, investigaciones recientes han comenzado a explorar enfoques que combinan la estructura de los métodos clásicos con la capacidad adaptativa del aprendizaje por refuerzo (Pfeiffer et al., 2017; Xie et al., 2018). En esta línea, el presente trabajo adopta un enfoque basado exclusivamente en DRL, específicamente el algoritmo SAC, entrenado en un entorno simulado de sucursal bancaria mediante Gazebo y ROS 2, con el objetivo de aprender una política de navegación autónoma que permita al robot visitar de forma secuencial múltiples puntos de interés dentro de la agencia.

2. Planteamiento del problema

Los entornos de servicio al público, como las agencias bancarias, presentan distribución espacial heterogénea y múltiples zonas de interés con funciones distintas. Un robot guía autónomo en este contexto debe navegar eficientemente hacia diferentes puntos tales como cajeros automáticos, ventanillas, áreas de atención sin depender de mapas actualizados manualmente ni de intervención humana.

Los sistemas de navegación basados en modelos cinemáticos estáticos presentan limitaciones cuando el comportamiento real del robot difiere del modelo asumido, situación frecuente por desgaste mecánico, variaciones en la superficie y retrasos en actuadores. Estas discrepancias se acumulan con el tiempo y degradan la navegación sin mecanismos de corrección en línea.

Este problema presenta tres desafíos específicos:

- Discrepancia modelo–realidad: el modelo cinemático idealizado no captura perturbaciones reales como derrapes, retrasos o asimetría de ruedas, introduciendo errores acumulativos en navegación prolongada, para mitigar la discrepancia entre el modelo cinemático idealizado y el comportamiento real del robot, se puede incorporar un modelo de dinámica residual aprendido, capaz de aproximar las perturbaciones no modeladas a partir de la experiencia.
- Aprendizaje multi-objetivo desbalanceado: en problemas multiobjetivos, los objetivos suelen encontrarse a distancias muy distintas del punto de spawn, para evitar el sesgo de aprendizaje hacia objetivos cercanos al punto de inicio, puede emplearse un curriculum dinámico basado en la dificultad de las tareas, donde la probabilidad de selección de objetivos se ajusta en función de su distancia o tasa de éxito histórica.
- Integración en tiempo real con ROS2: el entrenamiento en un sistema de robótica introduce latencias no deterministas que impiden la aplicación directa de algoritmos de RL diseñados para simuladores síncronos, La presencia de latencias no deterministas puede abordarse mediante estrategias de



desacoplamiento entre percepción, control y aprendizaje, utilizando buffers de experiencia y múltiples actualizaciones de red por interacción con el entorno.

3. Objetivos

3.1. Objetivo Principal

Desarrollar un modelo de aprendizaje incremental basado en aprendizaje por refuerzo profundo que permita a un robot móvil autónomo navegar en un entorno interior simulado representativo de una agencia bancaria, con el fin de guiar a los usuarios hacia puntos de interés específicos como cajeros automáticos, ventanillas y áreas de atención al cliente.

3.2. Objetivos Específicos

- Diseñar el entorno de simulación en ROS2 que represente una agencia bancaria con puntos de interés definidos, estableciendo el espacio de estados, acciones y la función de recompensa del agente.
- Determinar las características del espacio de estados y del modelo de movimiento del robot en el entorno simulado de la agencia bancaria y su impacto en la política de navegación aprendida.
- Diseñar una estrategia de aprendizaje incremental que permita actualizar online la política de navegación o el modelo de movimiento durante la exploración del entorno.
- Implementar el agente de navegación integrado con el modelo de movimiento de un robot en un entorno simulado mediante ROS2.
- Evaluar los resultados obtenidos en distintos mapas simulados para verificar la generalidad y adaptabilidad de la propuesta.

3.3. Alcance

El proyecto se limita al entorno simulado de una agencia bancaria implementado en Gazebo con ROS 2. El entorno contiene cuatro puntos de interés (POIs) fijos y etiquetados: (1) Cajero automático (ATM); (2)

Cajero automático (ATM); (3) Ventanillas de atención; (4) Área de servicio al cliente.

El robot empleado es un TurtleBot3 Waffle Pi simulado con modelo cinemático diferencial. El entorno incluye personas en movimiento en esta fase de evaluación; todos los obstáculos (paredes, mobiliario) son estáticos. La tarea consiste en que el robot visite los cuatro POIs en secuencia, partiendo desde una posición de spawn fija en cada episodio.

4. Estado del Arte y Marco teórico

4.1. Estado del arte

En los últimos años, la robótica móvil ha evolucionado de forma significativa hacia sistemas capaces de adaptarse en tiempo real a entornos desconocidos, dinámicos y no estructurados (Mukherjee et al., 2025). Este avance ha estado impulsado principalmente por el desarrollo del aprendizaje profundo(DL) y del aprendizaje por refuerzo profundo(DRL) (Quinones-Ramirez et al., 2023; Wang & Wang, 2024). No obstante, muchos de estos enfoques presentan altos costes computacionales y procesos de entrenamiento complejos (Rajawat et al., 2024), lo que limita su aplicabilidad en escenarios donde se requiere una adaptación rápida o donde los recursos de cómputo son reducidos (Yu, 2025).

Tradicionalmente, la exploración robótica se ha abordado mediante estrategias clásicas como la exploración basada en fronteras y los métodos basados en muestreo (Anuvarshini et al., 2025; İşleyen et al., 2025). La exploración basada en fronteras dirige al robot hacia la frontera entre regiones conocidas y zonas aún no exploradas con el objetivo de maximizar la adquisición de nueva información (Wang et al., 2024). Aunque este enfoque es sencillo y ampliamente utilizado, suele determinar los objetivos de exploración únicamente a partir del estado actual del robot, sin considerar mecanismos de adaptación a largo plazo (Hu et al., 2022). Por otro lado, algoritmos como Rapidly-exploring Random Trees (RRT) y Probabilistic Roadmaps (PRM) han demostrado ser eficaces para la planificación de trayectorias en entornos complejos y de alta dimensionalidad (Anuvarshini et al., 2025). Sin embargo, estos métodos suelen asumir modelos de movimiento fijos y carecen de capacidades de adaptación en tiempo real, lo que reduce su efectividad en escenarios dinámicos o previamente no encontrados (Chang et al., 2025).

Ante estas limitaciones, ha surgido un creciente interés por integrar el aprendizaje directamente durante la exploración. En este enfoque, los robots no se limitan a ejecutar trayectorias preprogramadas, sino que aprenden de su propia experiencia mientras interactúan con el entorno, ajustando su comportamiento en función de lo observado (Molina et al., 2022). Este tipo de aprendizaje resulta especialmente relevante en tareas en las que no se dispone de mapas previos ni de conocimiento a priori del entorno (Mukherjee et al., 2025). Por ejemplo, algunos trabajos han propuesto estrategias que maximizan la información obtenida durante la exploración mediante criterios basados en entropía, permitiendo a los robots inferir patrones de comportamiento humano y mejorar la navegación en entornos compartidos (Molina et al., 2022).

Dentro de esta línea, varios estudios han abordado el uso de modelos de coste aprendidos durante la exploración con el objetivo de mejorar la eficiencia y la seguridad del desplazamiento del robot (Warnakulasuriya et al., 2025). En terrenos irregulares o entornos parcialmente estructurados, los modelos de coste tradicionales resultan insuficientes, ya que no reflejan adecuadamente el impacto del entorno sobre el movimiento del robot. Para resolver este problema, se han propuesto enfoques basados en aprendizaje por refuerzo adaptativo que utilizan funciones de recompensa dinámicas para aprender trayectorias energéticamente eficientes sin requerir modelos previos del terreno (Warnakulasuriya et al., 2025).

Otro eje relevante de investigación es la navegación basada en políticas aprendidas, donde el robot emplea técnicas de aprendizaje por refuerzo profundo para tomar decisiones de navegación de forma autónoma (Wu et al., 2024). Estos enfoques han mostrado resultados prometedores al permitir que los robots aprendan y se adapten sin necesidad de grandes conjuntos de datos etiquetados, siendo capaces de operar en tiempo real sin intervención humana (Mukherjee et al., 2025; Wu et al., 2024). En este contexto, se han desarrollado marcos como Online Planner's Parameter Adaptation (OPPA), que combinan sistemas basados en reglas con aprendizaje para ajustar

dinámicamente los parámetros del planificador, mejorando la adaptabilidad y la seguridad (Chang et al., 2025). Este tipo de métodos ha sido aplicado con éxito en tareas como SLAM activo y evitación de obstáculos dinámicos (Cai et al., 2025; Nam et al., 2023).

El aprendizaje profundo también ha tenido un impacto notable en la exploración y navegación robótica. Las redes neuronales convolucionales se han utilizado para procesar información sensorial y apoyar tareas de control visual en tiempo real (Gubbi & Lediju Bell, 2021), permitiendo al robot reaccionar directamente a los datos de entrada sin necesidad de procesamiento intermedio complejo. Asimismo, han surgido enfoques de navegación sin mapas, en los que los robots exploran entornos desconocidos basándose únicamente en información sensorial, optimizando la reutilización de experiencias relevantes para mejorar el aprendizaje (Wang & Wang, 2024).

Ejemplos representativos de estos avances son los métodos basados en aprendizaje profundo y aprendizaje por refuerzo, como ARIADNE y CNN-RL (Cao et al., 2023). ARIADNE emplea aprendizaje por refuerzo profundo (DRL) para permitir a los robots móviles explorar y mapear rápidamente entornos desconocidos, replanificando trayectorias de forma online a medida que se actualiza la información disponible (Cao et al., 2023). Estos enfoques buscan superar las limitaciones de los planificadores clásicos basados en fronteras y muestreo (Cao et al., 2023). De forma similar, los algoritmos de DRL se han aplicado a la planificación de movimiento continuo, mapeando directamente las observaciones sensoriales a acciones continuas, lo que ofrece ventajas frente a enfoques discretos (Wu et al., 2024). El uso de Deep Q-Networks (DQN) también ha demostrado ser eficaz en tareas como acoplamiento y evitación de obstáculos, permitiendo el aprendizaje autónomo del robot (Lai et al., 2025).

La Figura 1 muestra un problema en la planificación de trayectorias de robots, donde un robot necesita desplazarse desde su posición actual hasta una

posición objetivo evitando obstáculos en un espacio bidimensional. Este desafío fundamental está siendo abordado activamente mediante métodos de aprendizaje por refuerzo profundo.

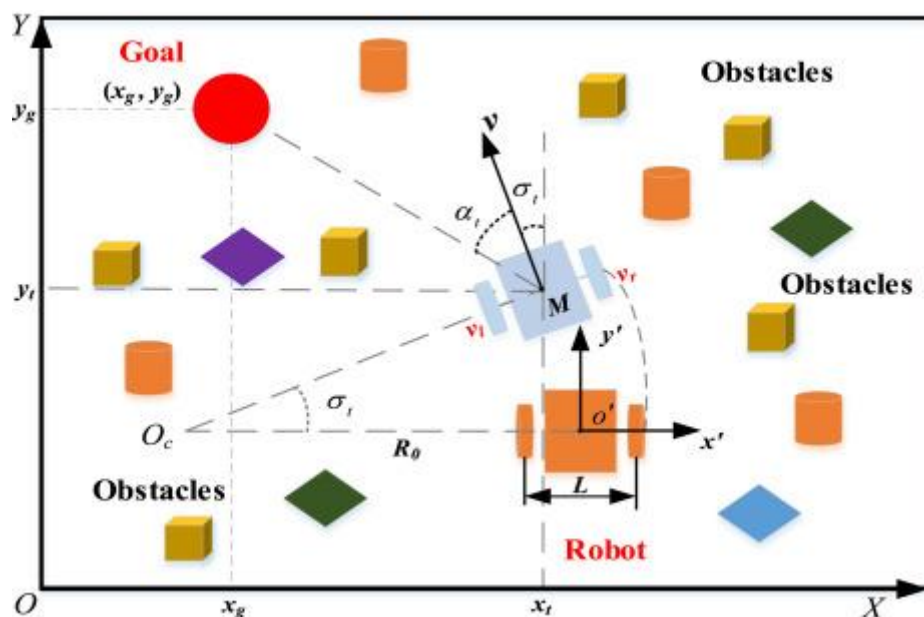


Figura 1. Problema típico en la planificación de trayectorias de robots. Tomado de (Sun et al., 2023).

Adicionalmente, en la actualidad ha cobrado relevancia el desarrollo de enfoques de aprendizaje online ligero y planificación adaptativa, orientados a lograr un equilibrio entre capacidad de adaptación y eficiencia computacional (Sarabakha et al., 2023). Estos métodos integran aprendizaje online y aprendizaje continuo, permitiendo a los robots generalizar conocimiento, adaptarse a condiciones variables y retener información previamente adquirida (Sarabakha et al., 2023). Técnicas como el Q-learning en tiempo real han demostrado ser eficaces para la planificación de trayectorias y la evitación de obstáculos dinámicos cuando se combinan con estrategias de exploración adecuadas (Kim & Lee, 2022). Asimismo, el control adaptativo de parámetros en tiempo real ha sido explorado mediante técnicas de DRL para ajustar el comportamiento del robot en función de múltiples objetivos (Sui & Chen, 2024).

En este contexto, los métodos de expansión informada ajustan la planificación en tiempo real basándose en nueva información del entorno (Sun et al., 2023). Entre ellos se incluyen enfoques que combinan control predictivo basado en modelos (MPC) con aprendizaje por refuerzo profundo para generar trayectorias libres de colisiones de forma eficiente (Zhang et al., 2023). También se han estudiado simplificaciones de la planificación en el espacio de creencias, basadas en procesos de decisión de Markov parcialmente observables que proporcionan un marco riguroso para la toma de decisiones bajo incertidumbre en tareas como SLAM activo y planificación informativa adaptativa (İşleyen et al., 2025). Por último, los métodos jerárquicos que combinan exploración basada en fronteras a alto nivel con planificadores de trayectorias basados en DRL a bajo nivel han demostrado mejorar la capacidad de adaptación en navegación y mapeo autónomos (Cai et al., 2025).

Con base en la revisión del estado del arte, se identifica que los robots autónomos en espacios públicos enfrenta cuatro brechas no resueltas: (1) la mayoría de sistemas DRL se validan en entornos genéricos, no en espacios reales con dinámicas humana; (2) no existen mecanismos efectivos para evitar olvido catastrófico en aprendizaje continuo multi-objetivo; (3) la adaptación incremental de modelos dinámicos rara vez se integra con políticas entrenadas bajo latencias ROS ($\approx 200\text{ms}$); (4) falta evaluación sistemática de agentes multi-objetivo en escenarios bancarios.

4.2. Marco Teórico

4.2.1. Introducción y fundamentos conceptuales

En el desarrollo de sistemas robóticos autónomos y socialmente responsables, resulta imprescindible que los agentes sean capaces de aprender y adaptarse de manera continua en entornos dinámicos y parcialmente desconocidos. Esta capacidad es especialmente relevante en escenarios reales, donde las condiciones operativas, los objetivos y las interacciones pueden variar de forma impredecible a lo largo del tiempo (Lesort et al., 2020).

El Aprendizaje Continuo se define como la capacidad de un agente para adquirir conocimiento de manera incremental a partir de flujos de datos continuos, preservando al mismo tiempo el conocimiento previamente adquirido. Por su parte, el Aprendizaje en Línea permite que los sistemas ajusten sus modelos durante la operación, sin requerir procesos de entrenamiento fuera de línea ni supervisión externa constante. En robótica, la combinación de estos enfoques suele estructurarse de la siguiente manera: el aprendizaje profundo proporciona capacidad de generalización, el aprendizaje en línea facilita la adaptación inmediata a nuevas condiciones y el aprendizaje continuo garantiza la retención del conocimiento a largo plazo (Sarabakha et al., 2023).

4.2.2 Fundamentos de la Robótica Móvil Autónoma

Los robots móviles autónomos se caracterizan por su capacidad de percibir el entorno, tomar decisiones y ejecutar acciones de manera independiente (Triviño & Marchesi, 2001). Desde una perspectiva funcional, su arquitectura se organiza típicamente en tres módulos principales: **percepción, planificación y control**. La percepción permite estimar el estado del robot y del entorno a partir de sensores; la planificación genera trayectorias o políticas de acción; y el control ejecuta

dichas decisiones sobre los actuadores físicos (Rosenblatt & Hendler, 1999). La Figura 2 ilustra esta arquitectura modular.

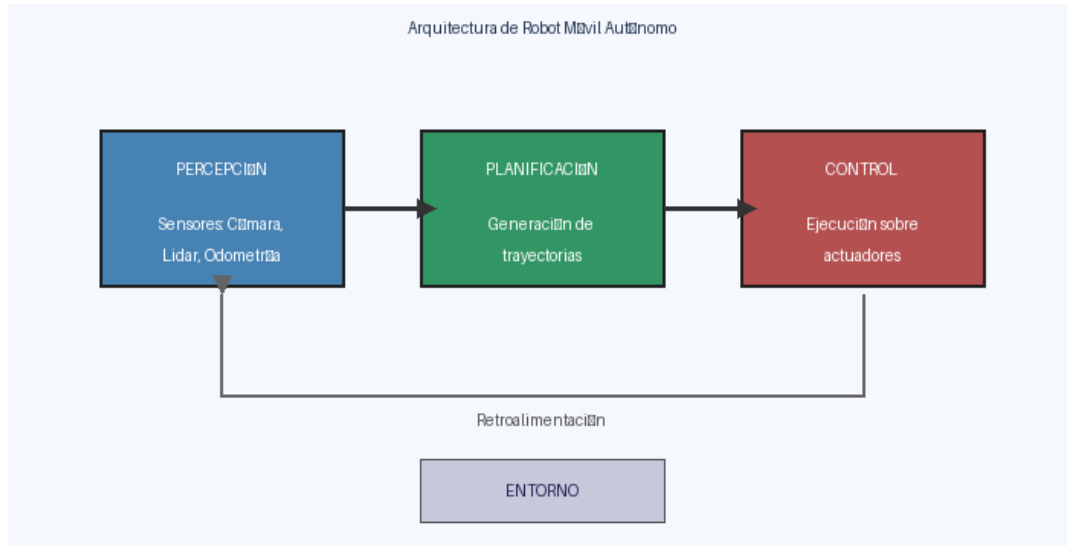


Figura 2. Arquitectura modular de percepción, planificación y control en robots móviles autónomos. Adaptado de Siegwart et al. (2011).

En entornos reales, estos módulos operan bajo condiciones de incertidumbre, tanto en las mediciones sensoriales como en la ejecución del movimiento. Esta incertidumbre motiva el uso de modelos aproximados y estrategias adaptativas, especialmente cuando el robot se despliega en escenarios desconocidos o dinámicos.

4.2.2.1. Modelos de Movimiento y Dinámica del Robot

El modelo de movimiento describe la relación entre las acciones de control aplicadas al robot y la evolución de su estado en el espacio. En robótica móvil, estos modelos se clasifican en modelos cinemáticos, que abstraen fuerzas y masas centrándose en la geometría del movimiento, y modelos dinámicos, que incorporan efectos físicos como inercia, fricción y fuerzas externas (Siegwart et al., 2011).

En la práctica, los robots operan con modelos idealizados que no capturan completamente las condiciones reales. Factores como el desgaste mecánico, variaciones en la superficie de contacto, retrasos en los actuadores y perturbaciones externas introducen discrepancias entre el modelo asumido y el comportamiento real. Los métodos de aprendizaje

incremental del modelo de movimiento permiten reducir progresivamente estos errores mediante la adaptación continua durante la ejecución.

4.2.2.2 Incertidumbre Sensorial y Estimación del Estado

La percepción robótica se basa en sensores que proporcionan observaciones parciales y ruidosas del entorno. Sensores como cámaras, LiDAR (Light Detection and Ranging), odometría y unidades inerciales están sujetos a errores sistemáticos y aleatorios, lo que genera incertidumbre en la estimación del estado del robot. Para gestionar esta incertidumbre, se emplean técnicas de estimación probabilística que integran información sensorial a lo largo del tiempo (Thrun et al., 2005).

La calidad de la estimación del estado impacta directamente en la planificación de trayectorias y en la toma de decisiones. En entornos no estacionarios, los modelos de percepción previamente entrenados pueden perder validez, lo que refuerza la necesidad de mecanismos de actualización continua.

4.2.3. Aprendizaje Supervisado, No Supervisado y por Refuerzo

Los sistemas robóticos pueden incorporar diferentes paradigmas de aprendizaje según el tipo de señal de retroalimentación disponible. El aprendizaje supervisado depende de datos etiquetados y es adecuado para tareas de percepción estática. El aprendizaje no supervisado permite descubrir estructuras latentes en los datos, útil para la representación del entorno. En contraste, el aprendizaje por refuerzo (RL) es especialmente apropiado para sistemas autónomos, ya que permite optimizar comportamientos a partir de la interacción directa con el entorno, sin requerir etiquetas explícitas (Sutton & Barto, 2018).

4.2.3.1. Fundamentos del Aprendizaje por Refuerzo en Robótica

El aprendizaje por refuerzo modela la interacción entre un agente y su entorno mediante un Proceso de Decisión de Markov (MDP), definido formalmente por la tupla (S, A, T, R) , donde S es el espacio de estados,

A el de acciones, T la función de transición y R la función de recompensa (Sutton & Barto, 2018). La Figura 2 representa el ciclo de interacción agente–entorno.

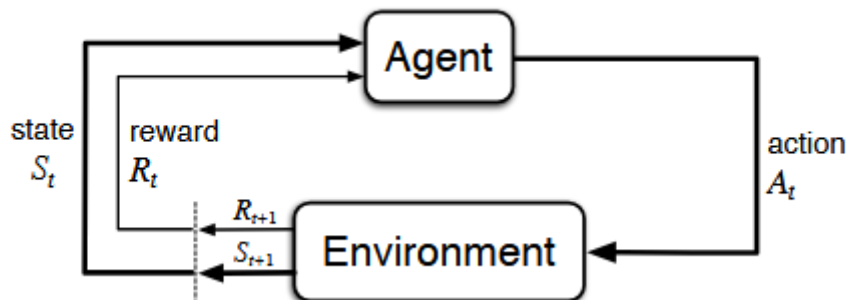


Figura 3. Ciclo de interacción agente–entorno en el Proceso de Decisión de Markov. Tomado de Sutton & Barto (2018).

Los enfoques de RL se clasifican en métodos *model-free*, que aprenden directamente de la experiencia sin modelo del entorno, y métodos *model-based*, que construyen un modelo interno para mejorar la eficiencia del aprendizaje. La aplicación de RL en robots reales presenta desafíos importantes: alto costo de muestreo, no estacionariedad del entorno y la necesidad de garantizar comportamientos seguros durante el aprendizaje.

4.2.4. Aprendizaje por Refuerzo Profundo para Navegación.

El aprendizaje profundo ha permitido extender el RL a espacios de estado y acción de alta dimensionalidad mediante redes neuronales profundas (LeCun et al., 2015). Estas técnicas facilitan el procesamiento directo de observaciones sensoriales complejas y la aproximación de funciones de valor o políticas de control.

A pesar de su capacidad de representación, los modelos profundos presentan dificultades relacionadas con la estabilidad del entrenamiento y la generalización fuera de los escenarios de entrenamiento. La integración del aprendizaje profundo con mecanismos de aprendizaje en línea y continuo resulta fundamental para mantener un desempeño robusto en

entornos cambiantes. A continuación, se presentan varios algoritmos y técnicas clave empleados en este trabajo.

4.2.4.1. Soft Actor-Critic (SAC)

El algoritmo **Soft Actor-Critic (SAC)** es un método de aprendizaje por refuerzo off-policy basado en el principio de entropía máxima, que maximiza simultáneamente la recompensa esperada y la entropía de la política. Esto promueve la exploración, mejora la robustez ante perturbaciones y evita la convergencia prematura a políticas subóptimas (Haarnoja et al., 2018). SAC es especialmente adecuado para espacios de acción continuos, como el control simultáneo de velocidad lineal y angular de un robot móvil. La Figura 4 ilustra la arquitectura del algoritmo.

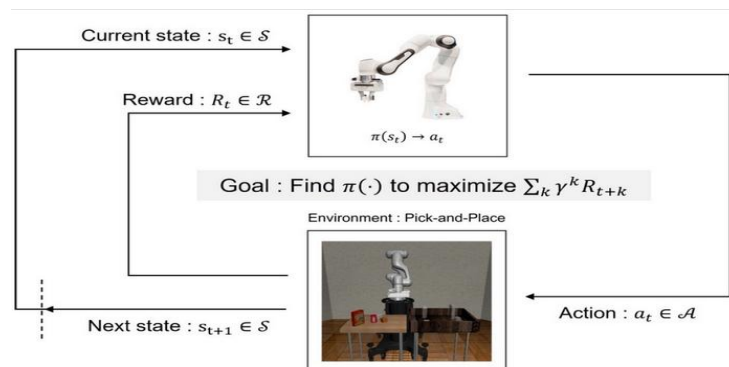


Figura 4. Arquitectura del algoritmo Soft Actor-Critic. Tomado de Haraguchi, K. et al. (2023).

La recompensa esperada acumulada si el agente toma la acción a en el estado s y luego sigue la política π .

$$Q^{\pi}(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1)$$

donde:

- s = estado
- a = acción
- r_t = recompensa
- γ = factor de descuento

En SAC, el valor Q se modifica para incluir entropía, quedando:

$$Q^{\pi}(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha H(\pi(\cdot|s_t))) \right] \quad (2)$$

El objetivo de SAC combina recompensa acumulada y entropía de la política mediante el parámetro de temperatura α , que pondera la importancia relativa de la exploración frente a la explotación. Una extensión relevante propuesta por Haarnoja et al. (2018) es el ajuste automático de α , que actualiza el parámetro de temperatura mediante gradiente descendente respecto a una entropía objetivo-predefinida, eliminando la necesidad de ajuste manual y mejorando la estabilidad del entrenamiento.

$$H(\pi(\cdot|s_t)) = -\log \pi(a_t|s_t) \quad (3)$$

En esta formulación, $H(\pi(\cdot|s_t))$ representa la entropía de la política en el estado s_t , mientras que α es el parámetro de temperatura que regula el equilibrio entre exploración y explotación.

Este término favorece políticas con mayor diversidad de acciones, lo que mejora la exploración del espacio de decisiones durante el entrenamiento.

4.2.4.2. Twin Critics y Actualización de Redes Objetivo

Para reducir la sobreestimación del valor Q estableciendo uno de los principales problemas de estabilidad en el aprendizaje por refuerzo

profundo (Mukherjee et al., 2025). SAC emplea dos redes críticas independientes Q_1 y Q_2 . El valor objetivo (target) utilizado para entrenar las redes Critic combina la recompensa inmediata, el valor futuro estimado y un término de regularización entrópica:

$$y_t = r_t + (1 - d_t) * \gamma [\min_{i=1,2} Q'_i(s_{t+1}, a_{\sim t+1}) - \alpha \log \pi(a_{\sim t+1} | s_{t+1})]$$

donde:

- y_t es el valor objetivo para actualizar los Critics Q_1 y Q_2 .
- r_t es la recompensa recibida en el paso t .
- d_t es el indicador de terminación del episodio (1 si terminó, 0 si continúa).
- γ es el factor de descuento, que evoluciona de 0.9 a 0.99 mediante *gamma annealing*.
- Q'_1, Q'_2 son las redes Critic objetivo (*target networks*), actualizadas suavemente con $\tau = 0.005$.
- $\min Q'_i$ selecciona el valor mínimo entre ambos Critics para mitigar la sobreestimación característica de Q-learning.
- s_{t+1} es el estado siguiente observado tras ejecutar la acción.
- $\tilde{a}_{t+1}$ es una acción muestreada de la política actual π en el estado siguiente.
- α es el coeficiente de temperatura que pondera la entropía, ajustado automáticamente durante el entrenamiento.
- $\log \pi(\tilde{a}|s)$ es la log-probabilidad de la acción bajo la política estocástica.

La estabilidad del entrenamiento se refuerza adicionalmente mediante la actualización suave de las redes objetivo (*soft target update*), que actualiza los parámetros de la red objetivo como una media exponencial móvil de los parámetros de la red activa, evitando cambios bruscos en el objetivo de entrenamiento.

4.2.4.3. Diseño de la Función de Recompensa

La función de recompensa es el componente central que guía el comportamiento del agente en el aprendizaje por refuerzo. Su diseño determina directamente qué comportamientos son incentivados y cuáles son penalizados. Una función de recompensa bien diseñada debe balancear la señal de progreso hacia el objetivo con penalizaciones por comportamientos indeseados como colisiones, trayectorias ineficientes o estancamiento (Quinones-Ramirez et al., 2023). La figura 5 representa de manera gráfica la función de recompensa durante la navegación autónoma del robot móvil. La superficie representa el valor de recompensa $R(x, y)$ en función de la posición del robot (x, y) en el espacio de trabajo. Los picos amarillos indican regiones de máxima recompensa (proximidad al objetivo), mientras que los valles azules representan regiones de mínima recompensa correspondientes a obstáculos, violaciones de seguridad y penalizaciones por estancamiento.

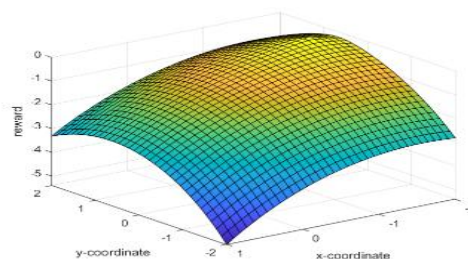


Figura 5. Componentes de una función de recompensa densa para navegación de robots móviles.
Tomado de Botteghi et al. (2020).

En tareas de navegación multi-objetivo, el escalado adaptativo de la recompensa según la dificultad de cada objetivo constituye una estrategia eficaz para mantener el aprendizaje balanceado. Al aumentar la magnitud de la señal de refuerzo para los objetivos con menor tasa de éxito histórica, se incentiva al agente a focalizarse en las áreas más desafiantes del entorno, evitando la especialización en tareas ya dominadas (Rajawat et al., 2024).

4.2.4.4. Curriculum Learning Adaptativo

El curriculum learning es una estrategia de entrenamiento que ordena o selecciona las tareas presentadas al agente según su dificultad, con el objetivo de acelerar el aprendizaje y mejorar la generalización. A diferencia de los enfoques con curriculum fijo, el curriculum dinámico ajusta la selección de tareas en función del rendimiento observado del agente durante el entrenamiento (Narvekar et al., 2020).

En robótica de navegación, el curriculum adaptativo permite que el agente practique más frecuentemente los objetivos con menor tasa de éxito, evitando el sobreajuste a tareas fáciles y manteniendo el aprendizaje activo en todo el rango de dificultad del entorno. La probabilidad de selección de cada tarea es inversamente proporcional al rendimiento histórico del agente en dicha tarea (Rajawat et al., 2024).

El efecto del curriculum learning en el proceso de entrenamiento puede analizarse a partir del comportamiento del rendimiento del agente a lo largo del tiempo. En particular, la organización progresiva de las tareas permite mejorar el desempeño inicial del agente, acelerar la velocidad de aprendizaje y favorecer una convergencia más estable. La Figura 6 ilustra estos efectos mediante la comparación del comportamiento del aprendizaje cuando se utiliza transferencia de conocimiento frente a un entrenamiento sin transferencia, destacando métricas como el rendimiento inicial (jumpstart), el tiempo necesario para alcanzar un umbral de desempeño (time to threshold) y el rendimiento final alcanzado (asymptotic performance).

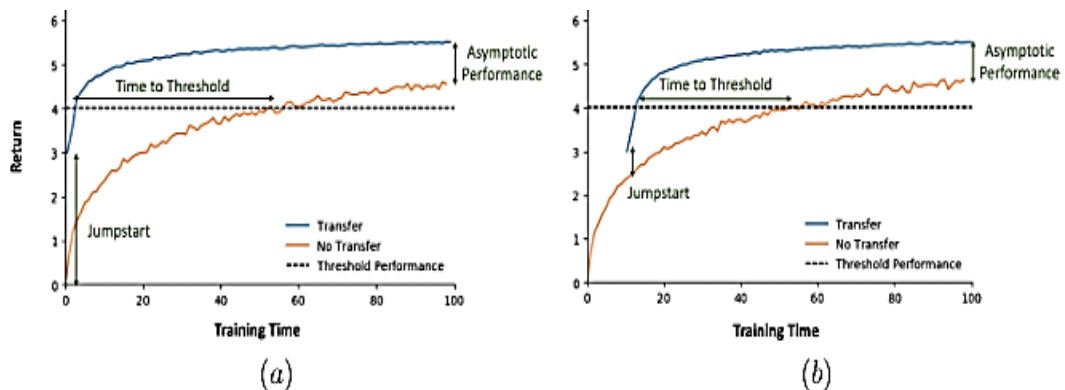


Figura 6. Comportamiento del aprendizaje durante el entrenamiento del agente, mostrando las métricas jumpstart, time to threshold y asymptotic performance. Adaptado de Narvekar et al. (2020).4.2.5. El Problema del Olvido Catastrófico

El olvido catastrófico es un fenómeno ampliamente reconocido en el aprendizaje continuo, que ocurre cuando la adquisición de nuevo conocimiento interfiere y degrada el conocimiento previamente aprendido (McCloskey & Cohen, 1989; Kirkpatrick et al., 2017). La Figura 7 ilustra este fenómeno comparando el rendimiento de un sistema sin mitigación frente a uno que emplea la técnica de *Elastic Weight Consolidation (EWC)*: las elipses representan regiones del espacio de parámetros que producen bajo error para las tareas A y B. El punto θ_a corresponde a los parámetros obtenidos tras entrenar en la tarea A. Al aprender la tarea B, un entrenamiento sin regularización puede desplazar los parámetros fuera de la región óptima de la tarea A, produciendo olvido catastrófico. La regularización L2 penaliza los cambios respecto a los parámetros originales, pero trata todos los parámetros de forma uniforme. En contraste, EWC penaliza únicamente los cambios en parámetros importantes para la tarea anterior, permitiendo encontrar una solución que preserve el desempeño en ambas tareas.

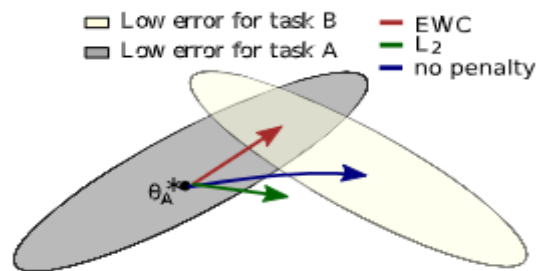


Figura 7. Degradación del rendimiento por olvido catastrófico y su mitigación mediante EWC. Tomado de Kirkpatrick et al. (2017).

En sistemas robóticos, el olvido catastrófico afecta directamente a los módulos de percepción y localización que se actualizan de manera continua. En el contexto del Aprendizaje por Refuerzo Continuo (CRL), este fenómeno se ve agravado por la naturaleza secuencial de la toma de decisiones y el gran espacio estado–acción involucrado. Estudios previos han demostrado que técnicas como la repetición de memoria (memory replay) y la consolidación de pesos constituyen estrategias eficaces para mitigar este problema (Lesort et al., 2020).

4.2.6. Gazebo como simulador para aprendizaje por refuerzo

Gazebo es un simulador robótico tridimensional de código abierto introducido por Koenig y Howard (2004) como respuesta a las limitaciones de los simuladores 2D existentes. Mantenido por la Open Source Robotics Foundation, evolucionó desde una arquitectura monolítica hasta la versión moderna (2022) basada en Entity Component System con soporte para múltiples motores de física (ODE, Bullet, DART) y renderizado OGRE. Su integración con ROS 2 a través del paquete gazebo_ros_pkgs hace que los topics sensor de escaneo (/scan), odometría (/odom) y comando de velocidad (/cmd_vel) sean transparentes entre el robot simulado y el físico, propiedad fundamental para el entrenamiento de agentes de RL

cuyas políticas deben transferirse a hardware sin modificar la capa de comunicación (Salimpour et al., 2025). La reproducibilidad determinista de episodios, el control programático del entorno y la posibilidad de acelerar la simulación hasta 5× la consolidan como la plataforma predominante en investigación de RL robótico (Zamora et al., 2016; Gonzalez Lopez et al., 2019). Su principal limitación es la brecha sim-to-real: discrepancias en fricción, inercia sensorial y latencias degradan las políticas al transferirlas a hardware (Zhao et al., 2020); en el presente TFM esta brecha se mitiga mediante un MotionModel que aprende en línea las desviaciones entre el modelo cinemático nominal y el comportamiento real del robot.

4.2.7. TurtleBot3 Waffle Pi: plataforma robótica de referencia

TurtleBot3 es una plataforma robótica móvil de código abierto desarrollada por ROBOTIS y Open Robotics, diseñada para investigación en SLAM y navegación autónoma (ROBOTIS, 2023). El modelo Waffle Pi seleccionado incorpora actuadores Dynamixel XM430-W210-T, una IMU de 9 ejes y un LiDAR omnidireccional LDS-02 de 360° con alcance de 3,5 m. Su cinemática diferencial no holonómica reduce el control a un vector continuo $[v, \omega]$, directamente compatible con el algoritmo SAC (Haarnoja et al., 2018). La Figura 8 indica todos los componentes de turtleBot3.

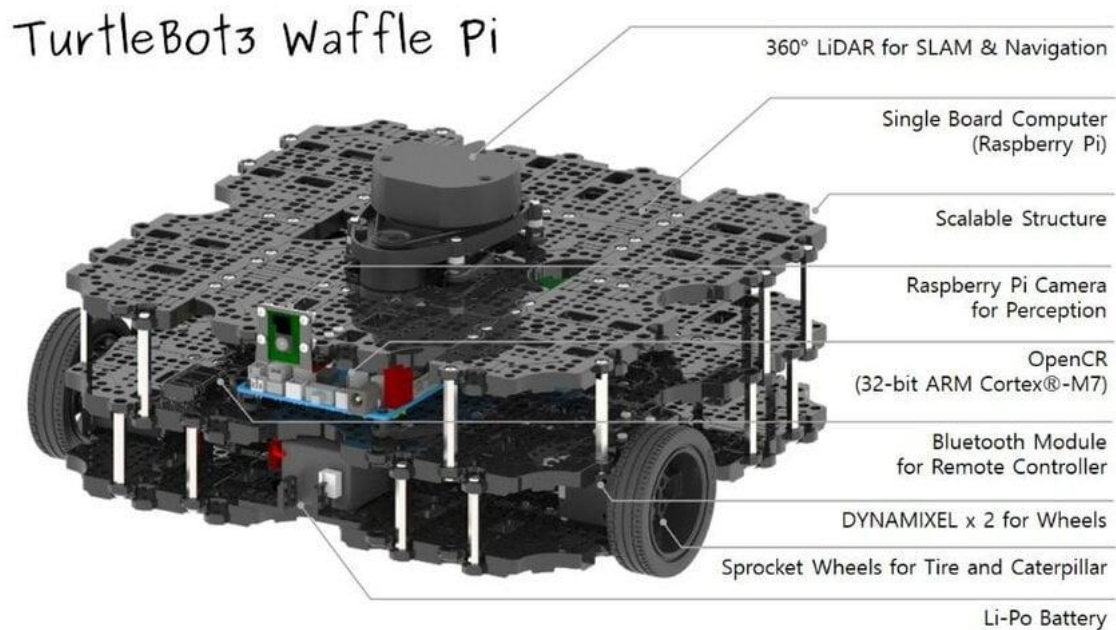


Figura 8. Componentes de TurtleBot3 waffle pi. Tomado de T Y Lee et al 2019

La variable v representa la velocidad lineal del robot, responsable del movimiento hacia adelante o atrás sobre su eje longitudinal. Se mide en metros por segundo (m/s). Valores positivos indican avance, valores negativos retroceso y $v = 0$ ausencia de desplazamiento lineal. La variable ω representa la velocidad angular del robot, que controla la rotación sobre su eje vertical. Se mide en radianes por segundo (rad/s). Valores positivos producen giro a la izquierda, valores negativos giro a la derecha y $\omega = 0$ indica que el robot no rota.

4.2.8. Aprendizaje en Línea de Final Abierto

Las metodologías de percepción visual autónoma basadas en aprendizaje en línea de final abierto pueden clasificarse según el tipo de conocimiento adquirido durante la operación. Estas incluyen: (a) el aprendizaje incremental de instancias, (b) la evolución dinámica de características, (c) el aprendizaje incremental de clases y tareas, y (d) los enfoques paralelos y distribuidos orientados al manejo de grandes volúmenes de datos (Sarabakha et al., 2023). El aprendizaje en línea se

implementa típicamente mediante arquitecturas duales que combinan una memoria episódica de corto plazo usualmente un buffer de experiencias recientes con mecanismos de memoria a largo plazo que preservan el conocimiento consolidado. (Lesort et al., 2020; Richert et al., 2008).

4.2.9. Métricas de Evaluación en Navegación Robótica

La evaluación del rendimiento de agentes de navegación robótica requiere métricas que capturen múltiples dimensiones del comportamiento: eficiencia de movimiento, cumplimiento de objetivos y seguridad operativa (Xiao et al., 2022). Estas métricas permiten comparar distintas políticas y determinar si un agente ha alcanzado un nivel de competencia aceptable para su despliegue.

Eficiencia de Trayectoria

La eficiencia de trayectoria (TE, Trajectory Efficiency) compara la distancia real recorrida por el robot con la distancia óptima teórica, definida como la suma de segmentos en línea recta entre puntos consecutivos (Mirowski et al., 2017):

$$TE = \left(\frac{d_{opt}}{d_{real}} \right) \times 100$$

donde:

d_{opt} es la suma de distancias euclidianas en línea recta entre objetivos consecutivos.

d_{real} es la distancia total acumulada por el robot durante el episodio.

Valores cercanos a 100% indican trayectorias casi óptimas. En entornos con obstáculos, valores ligeramente inferiores son esperables debido a las maniobras de evasión necesarias.

Tasa de Éxito

La tasa de éxito (SR, Success Rate) mide la proporción de episodios en que el agente completa todos los objetivos requeridos. Para tareas multi-objetivo secuenciales con N objetivos (Anderson et al., 2018):

$$SR = \left(\frac{1}{M}\right) \sum_{i=1}^M \mathbb{I}(g_i = N) \times 100$$

donde:

M es el número total de episodios de evaluación

g_i es el número de objetivos completados en el episodio i

$\mathbb{I}(\cdot)$ es la función indicadora: 1 si la condición se cumple, 0 en caso contrario

N es el número total de objetivos requeridos para considerar el episodio exitoso

Tasa Libre de Colisión

La tasa libre de colisión (CFR, Collision-Free Rate) mide la proporción de episodios sin ninguna colisión detectada, siendo fundamental en aplicaciones de servicio donde la seguridad es prioritaria (Kahn et al., 2021):

$$CFR = \left(\frac{1}{M}\right) \sum_{i=1}^M \mathbb{I}(c_i = 0) \times 100$$

donde:

c_i es el número de colisiones detectadas en el episodio i .

Ambas métricas (SR y CFR) deben reportarse conjuntamente, ya que un agente que alcanza todos los objetivos colisionando sistemáticamente no constituye una solución válida para entornos de servicio con presencia humana.

5. Metodología

Se utilizó una metodología de investigación aplicada, orientada al diseño, implementación y evaluación de un sistema de navegación autónoma basado en aprendizaje por refuerzo profundo para un robot móvil en un entorno interior simulado. El objetivo es que el robot aprenda a visitar cuatro puntos de atención al cliente en secuencia, adaptándose en tiempo real a las condiciones del entorno sin intervención humana.

La tarea se modela formalmente como un Proceso de Decisión de Markov (PDM) definido por la tupla:

$$MDP = (S, A, T, R) \quad (3)$$

donde:

S es espacio de estados continuo

A es espacio de acciones continuo bidimensional

T es función de transición inducida por el simulador Gazebo/ROS 2

R es función de recompensa multi-objetivo

El desarrollo se organizó en cuatro etapas secuenciales, descritas en la Tabla 1.

Tabla 1
Etapas del desarrollo del proyecto

Etapa	Denominación	Descripción
1	Diseño del entorno de simulación	Definición del espacio de estados, acciones y estructura física del entorno bancario en Gazebo/ROS 2
2	Implementación del agente	Diseño del agente SAC con actor estocástico, twin critics, ajuste automático de α y modelo de movimiento incremental.
3	Integración y entrenamiento	Configuración del curriculum progresivo, gamma annealing y buffers de experiencia
4	Evaluación y análisis	Medición de eficiencia de trayectoria, tasa de éxito y métricas de convergencia

5.1. Etapa 1: Diseño del entorno de simulación

El entorno de simulación define las condiciones bajo las cuales el agente aprende. Su diseño determina qué información percibe el robot, qué acciones puede ejecutar y cómo se estructura físicamente el espacio de navegación. Se implementa utilizando el simulador Gazebo integrado con ROS y contiene cuatro puntos de interés (POIs) fijos que el robot debe visitar en secuencia.

5.1.1. Representación del Estado

El robot necesita información suficiente para tomar decisiones de navegación seguras y orientadas al objetivo. Por este motivo, el vector de estado $s_t \in \mathbb{R}^{370}$ integra tres tipos de información: sensorial, propioceptiva y de progreso de la tarea. La Tabla 2 detalla la composición del vector de estado según la implementación.

Tabla 2

Composición del vector de estado (370 dimensiones)

Componente	Dimensiones	Rango	Descripción
Lecturas LiDAR	360	[0, 1]	lidar / LIDAR_MAX (alcance 3.5 m)
Posición (x, y)	2	[-1, 1]	x/20, y/20 (normalizado al entorno)
Orientación (yaw)	1	[-1, 1]	yaw / π
Distancia al POI actual	1	[0, 1]	curr_dist / 20
Ángulo al POI actual	1	[-1, 1]	curr_ang / π
Distancia al POI final	1	[0, 1]	final_dist / 20
Ángulo al POI final	1	[-1, 1]	final_ang / π
goal_progress	1	[0, 1]	len(goals_completed) / n_total

time_pressure	1	[0, 1]	step_count max_steps_ep	/
goals_remaining	1	[0, 1]	len(remaining) n_total	/
TOTAL	370	—	—	

La información sensorial proviene del LiDAR de 360°, cuyas 360 lecturas se normalizan al rango [0, 1] con un alcance máximo de 3.5 m. Proporciona al agente conciencia del entorno inmediato y es la fuente primaria para la evasión de obstáculos. La información propioceptiva incluye posición (x, y) normalizada respecto al tamaño del entorno (20 metros) y orientación (yaw) normalizada respecto a π . También incluye la distancia y el ángulo relativo tanto al POI activo como al POI final de la secuencia, todas normalizadas. Permite que el agente sepa dónde está, hacia qué dirección debe dirigirse inmediatamente y cuál es su destino final.

La información de progreso incluye tres variables: goal_progress, que indica qué fracción de los POIs se han completado en el episodio; time_pressure, que refleja qué fracción del tiempo máximo se ha consumido; y goals_remaining, que indica la proporción de POIs pendientes respecto al total. Adicionalmente, el estado incluye información sobre el POI final de la secuencia (distancia y ángulo relativos), siguiendo la metodología de Grillotti et al. (2025) que demostró que condicionar solo en el objetivo actual crea un POMDP, mientras que incluir el objetivo final permite planificar trayectorias compatibles con el futuro. Estas variables brindan observabilidad completa a la política, indicando cuánto queda por hacer, con cuánto tiempo disponible y hacia dónde debe dirigirse finalmente; información indispensable para planificar en una tarea multi-objetivo secuencial (Mehta et al., 2024).

5.1.2. Espacio de Acciones

El espacio de acciones define todas las acciones posibles que un agente de aprendizaje por refuerzo puede ejecutar sobre el entorno en cada instante de tiempo. En este caso el espacio de acciones es continuo y bidimensional:

$$a_t = (v_t, \omega_t)$$

(4)

donde:

$v_t \in [0, 0.26]$ m/s es velocidad lineal, restringida al avance (sin retroceso)

$\omega_t \in [-1.82, 1.82]$ rad/s es velocidad angular

La restricción de no retroceso reduce el espacio de búsqueda de la política y es coherente con el comportamiento de robots de servicio en espacios públicos

5.1.3. Entorno Simulado y Puntos de Interés

El entorno replica una agencia bancaria de 20×20 metros (400 m^2) con cuatro POIs que representan zonas de atención al cliente. El espacio se delimita por paredes ubicadas en ± 10 metros en ambos ejes y se representa internamente en una cuadrícula donde cada celda corresponde a $10 \times 10 \text{ cm}$ para el cálculo de métricas de cobertura espacial. La Figura 8, 9, 10 y 11 muestran el entorno y los POIs.

Entorno simulado

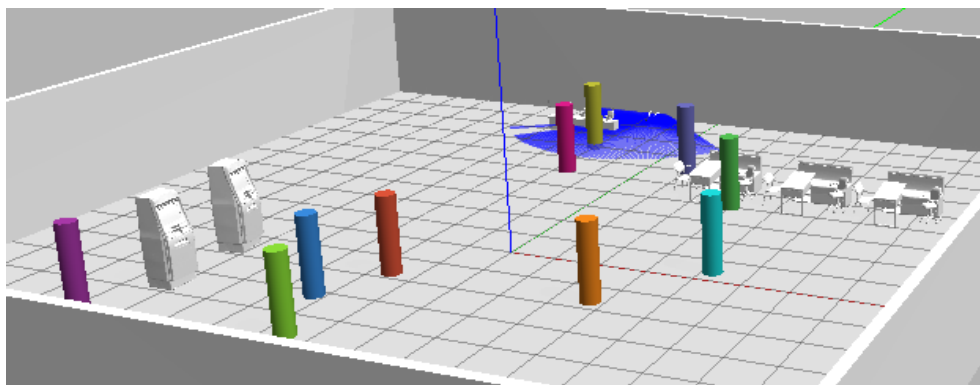


Figura 9. Entorno bancario simulado en Gazebo.

Objetivos

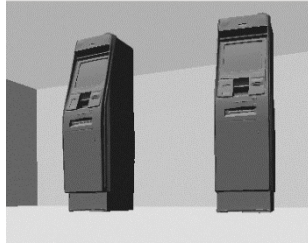


Figura 10. ATM 1 y ATM 2

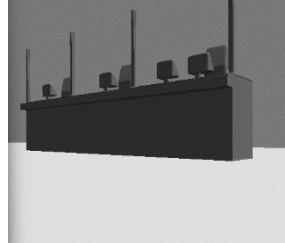


Figura 11. Ventanillas.

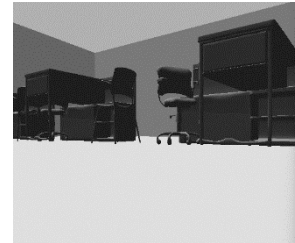


Figura 12. Servicio al cliente.

Los POIs están distribuidos a distancias crecientes desde el punto de inicio del robot, como se detalla en la Tabla 3.

Tabla 3

Puntos de interés del entorno bancario simulado

POI	Etiqueta	Coord. simulador (m)	Dist. al spawn (m)	Radio llegada (m)
1	ATM 1	(-5.12, -3.58)	~1.5	0.50
2	ATM 2	(-5.29, -1.58)	~3.5	0.50
3	Servicio al Cliente	(4.51, 4.66)	~10.7	0.50
4	Ventanilla	(-2.99, 8.19)	~13.5	0.50

Nota. Coordenadas en el sistema de referencia de Gazebo. El spawn corresponde a la posición inicial del robot (0.0, -5.0) m. El radio de llegada define la distancia umbral (en metros) para considerar que el robot ha alcanzado el punto de interés (POI).

En cada episodio el robot debe visitar los cuatro POIs en secuencia, mientras navega en un entorno que incluye hasta 10 personas simuladas como obstáculos dinámicos. Cada persona se modela como un cilindro de radio 0.2 m y altura 1.7 m, moviéndose a velocidades de hasta 0.3 m/s con evasión de obstáculos y pausas aleatorias que simulan comportamiento humano realista. El episodio concluye al completar la secuencia, al superar el límite de pasos (500 pasos por cada POI objetivo, es decir, hasta 2000 pasos para los 4 POIs), o al detectarse una colisión ($d_{\text{LiDAR}} < 0.15$ m).

5.2. Etapa 2: Implementación del Agente

El agente integra tres componentes diseñados para resolver los tres desafíos identificados: la política Soft Actor-Critic (SAC) para aprendizaje en espacios

continuos, el MotionModel para corregir la discrepancia modelo–realidad, y la función de recompensa para guiar el comportamiento multi-objetivo.

5.2.1. Política de Navegación: Soft Actor-Critic

Se adopta SAC, como algoritmo de aprendizaje por refuerzo. La elección se justifica por tres propiedades: es un algoritmo off-policy, lo que permite reutilizar experiencias pasadas y mejora la eficiencia de muestreo; opera sobre espacios de acción continuos, necesario para el control de velocidad del robot; e incorpora maximización de entropía, lo que mantiene la exploración activa durante todo el entrenamiento y evita convergencia prematura a estrategias subóptimas.

Para reducir la sobreestimación del valor de las acciones, el agente emplea dos redes de valor independientes (twin critics). Cada red crítica es un perceptrón multicapa (MLP) que recibe como entrada la concatenación del estado (370 dimensiones) y la acción (2 dimensiones: v , ω), totalizando 372 dimensiones, y produce como salida una estimación escalar del valor $Q(s, a)$. En cada actualización se usa el mínimo de las dos estimaciones (clipped double-Q), lo que estabiliza el aprendizaje. La Tabla 4 detalla la arquitectura de cada red.

Tabla 4

Arquitectura de las redes neuronales del agente SAC

Red	Entrada	Arquitectura	Salida	Activación	Función
Actor	370	370→256→256→256	2×2 (μ , $\log_{\sigma}$)	ReLU + tanh	Genera $\pi(a s)$
Critic Q1	372	372→256→256→256→1	1 escalar	ReLU	Estima $Q_1(s,a)$
Critic Q2	372	372→256→256→256→1	1 escalar	ReLU	Estima $Q_2(s,a)$
Critic Target	372	372→256→256→256→1	1 escalar	ReLU	Calcula targets Q

Nota. Entrada del Critic Q1 y Q2: concatenación de estado (370) y acción (2) = 372 dimensiones. Inicialización Xavier uniforme en todas las redes. La actualización de las redes objetivo sigue la media exponencial móvil con $\tau = 0.005$.

5.2.2. Modelo de Movimiento Incremental

Para abordar la discrepancia entre el modelo cinemático nominal y la dinámica real del robot, el sistema incorpora un MotionModel que aprende en línea cómo el robot responde realmente a cada acción, este componente se actualiza continuamente con las transiciones observadas durante el entrenamiento.

El MotionModel aprende a predecir el cambio de estado que producirá ejecutar la acción a_t en el estado s_t :

$$\Delta \hat{s}_t = f_{\theta(s_t, a_t)} \quad (6)$$

donde:

f_{θ} es red neuronal del MotionModel. Véase la Tabla 3

$\Delta \hat{s}_t$ es cambio de estado predicho; el estado siguiente se estima como $\hat{s}_{t+1} = s_t + \Delta \hat{s}_t$

Durante el curriculum progresivo, el MotionModel enfrenta un riesgo al avanzar a las diferentes fases ya que al tener 1 a 4 POIs activos el modelo se expone a nuevas dinámicas y puede olvidar lo aprendido en fases anteriores, fenómeno conocido como olvido catastrófico. Para prevenirlo, se incorpora Elastic Weight Consolidation (EWC, Kirkpatrick et al., 2017).

EWC funciona como una memoria selectiva: identifica qué parámetros del modelo fueron más importantes para predecir correctamente en episodios anteriores y penaliza los cambios sobre ellos al adaptarse a nuevas condiciones. La pérdida total del MotionModel combina error de predicción y regularización:

$$L_{motion} = MSE(f_{\theta(s_t, a_t)}, \Delta s_t) + L_{EWC} \quad (7)$$

donde:

MSE es error cuadrático medio sobre transiciones recientes del model_buffer

L_{EWC} es penalización EWC:

$$\left(\frac{\lambda}{2}\right) \sum_i \hat{F}_i (\theta_i - \theta^*_{i})^2, \text{ con } \lambda = 400 \quad (8)$$

θ^*_{i} es parámetros óptimos consolidados al final del episodio anterior

Un $\lambda = 400$ actúa como memoria selectiva de intensidad alta: preserva el conocimiento consolidado y permite adaptación a condiciones nuevas. La Tabla 5 resume la configuración completa del componente.

Tabla 5

Configuración del MotionModel

Componente	Configuración
Arquitectura	MLP: $372 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 370$
Activación	SiLU (Sigmoid Linear Unit) en capas ocultas
Actualización	Cada 50 pasos; 3 iteraciones $\times$ batch de 64 transiciones
Buffer	5 000 transiciones FIFO
EWC λ	400 (consolidación al fin de cada episodio)

Nota. FIFO = primero en entrar, primero en salir. La consolidación se ejecuta al final de cada episodio si el `model_buffer` contiene al menos 100 transiciones.

5.2.3. Función de Recompensa Multi-objetivo

La función de recompensa traduce el objetivo de la tarea el cual se basa en visitar cuatro POIs en secuencia de forma segura y eficiente, en una señal numérica que guía el aprendizaje. Esta señal combina un componente de llegada y un componente de paso ordinario. Cuando el robot alcanza el POI activo, recibe un bonus que crece con cada objetivo completado:

$$r_{goal(n)} = 100 + 100 \cdot n \quad (9)$$

donde:

n es número de POIs completados hasta ese momento en el episodio

El escalado compensa el efecto del descuento temporal acumulado: los POIs lejanos se alcanzan tarde en el episodio, momento en que γ^T reduce su valor percibido. Un bonus creciente corrige este desequilibrio y mantiene la motivación del agente para completar la secuencia completa (Mehta et al., 2024). Al finalizar los cuatro POIs se añaden 200 puntos adicionales como incentivo de tarea completa. Para los pasos ordinarios, la función combina incentivos de progreso con penalizaciones de seguridad y eficiencia.

$$r_{goal}(n) = 100 + 100 \cdot n + 300 \cdot I(n = N) \quad (10)$$

Donde:

- n es número de POIs completados
- N es total de POIs (4)
- $I(n = N)$ es función indicadora (vale 1 si $n = N$, 0 en caso contrario)

El **retorno descontado** (también llamado *discounted return* o *cumulative discounted reward*) es la suma de todas las recompensas futuras, donde cada recompensa se multiplica por un factor de descuento γ (gamma) elevado al número de pasos en el futuro

$$G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k} \quad (11)$$

donde γ evoluciona de 0.9 a 0.99 durante 200,000 pasos (gamma annealing). γ^T reduce el valor percibido de recompensas distantes, lo que se compensa con el escalado progresivo.

5.3. Etapa 3: Integración y Entrenamiento

Esta etapa describe cómo los componentes anteriores se combinan en un proceso de entrenamiento coherente y cómo se diseñaron dos mecanismos

currículum progresivo y factor de descuento adaptativo para hacer ese proceso progresivo y estable. La Figura 13 muestra la arquitectura de integración del sistema de entrenamiento, donde el Entorno genera estados s_t y ejecuta acciones, alimentando simultáneamente al Replay Buffer y al Model Buffer. El Actor recibe el estado y genera acciones mediante la política $\pi(a|s)$, mientras que los Twin Critics (Q_1, Q_2) estiman valores Q a partir de la concatenación $[s_t, a_t]$. El módulo SAC UPDATE realiza dos actualizaciones por paso (replay ratio = 2) con lotes de 256 muestras, actualizando tanto al Actor como a los Critics. Las redes objetivo se actualizan suavemente con $\tau = 0.005$. Paralelamente, el MotionModel se entrena con lotes de 64 del Model Buffer, aplicando consolidación EWC ($\lambda = 400$) al final de cada episodio para mitigar el olvido catastrófico durante las transiciones de fase del currículum.

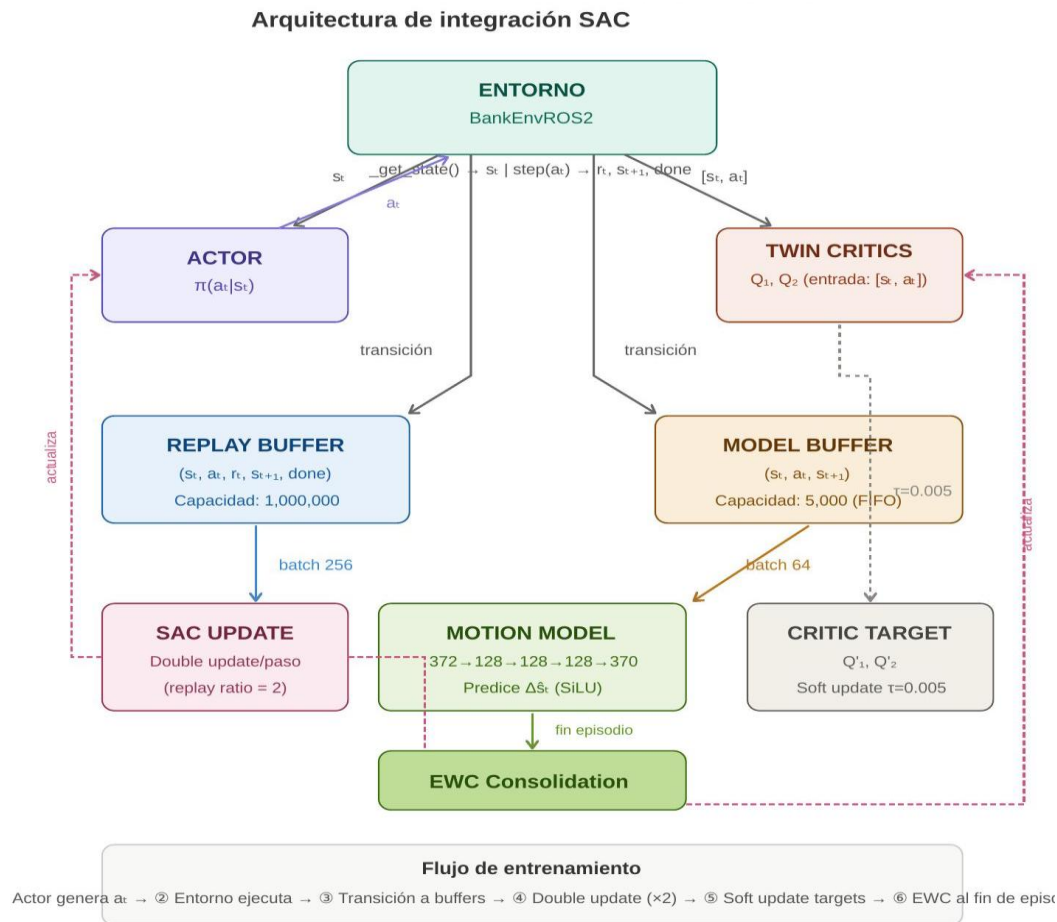


Figura 13 Arquitectura de integración del sistema de entrenamiento SAC.

El sistema emplea dos estructuras de almacenamiento complementarias que sirven propósitos distintos durante el entrenamiento. Estas estructuras almacenan experiencias del agente para entrenar modelos y políticas. El Replay Buffer principal almacena un millón de transiciones para permitir el entrenamiento off-policy, donde el agente aprende de experiencias pasadas sin requerir que provengan de la política actual. El Model Buffer mantiene solo las 5,000 transiciones más recientes; esta restricción temporal asegura que el MotionModel se adapte a las dinámicas actuales del robot. FIFO garantiza actualización continua con datos recientes. A continuación, se describe en la Tabla 6.

Tabla 6

Estructuras de almacenamiento de experiencias

Buffer	Capacidad	Contenido	Propósito
Replay buffer principal	1,000,000	(s, a, r, s', done)	Entrenamiento off-policy estable de SAC
Model buffer	5,000 (FIFO)	(s, a, s')	Entrenamiento del MotionModel con datos recientes

Nota. FIFO = First In, First Out. s = estado, a = acción, r = recompensa, d = done. El model_buffer se consolida con EWC al final de cada episodio.

5.3.1. Curriculum Progresivo

Entrenar directamente sobre la tarea completa, es decir, que aprenda a encontrar cuatro POIs a distancias crecientes desde el primer episodio es ineficiente, ya que el agente recibe señales de recompensa muy escasas al principio, lo que ralentiza el aprendizaje. En su lugar, se adopta un curriculum progresivo (Luo et al., 2023) que aumenta gradualmente la complejidad en cuatro fases: el agente aprende primero a navegar hacia 1 POI, luego 2, luego 3, y finalmente los 4.

El avance entre fases es automático: cuando la tasa de éxito sobre una ventana deslizante de 30 episodios supera el 70%, el sistema incrementa el número de POIs activos. Este umbral garantiza que el agente haya consolidado la fase actual antes de enfrentar la siguiente. La estrategia permite un aprendizaje denso en señal de recompensa desde el inicio, construyendo competencia de forma acumulativa.

5.3.2. Factor de Descuento Adaptativo

El factor de descuento “ γ ” determina cuánto peso da el agente a las recompensas futuras frente a las inmediatas. Un “ γ ” bajo al inicio facilita la estabilidad, pero un “ γ ” alto al final es necesario para planificar correctamente las trayectorias largas que requiere visitar cuatro POIs en secuencia.

Para obtener ambas ventajas, γ se incrementa linealmente de 0.9 a 0.99 durante los primeros 200 000 pasos de entrenamiento (Amit et al., 2020):

$$\gamma(t) = \gamma_{start} + \min\left(\frac{t}{T_{anneal}}, 1\right)(\gamma_{end} - \gamma_{start}) \quad (12)$$

donde:

t = Pasos de entrenamiento acumulados.

$T_{anneal} = 200\ 000$ es duración del annealing.

$\gamma_{start} = 0.9$ es valor inicial, favorece estabilidad temprana.

$\gamma_{end} = 0.99$ es valor final, permite optimizar trayectorias largas.

Esta progresión está sincronizada con el curriculum, es decir, cuando el agente comienza a aprender tareas más largas en fases avanzadas, y ya se encuentra en valores altos que valoran correctamente las recompensas distantes.

5.2.3. Hiperparámetros del Sistema

La Tabla 7,8 y 9 presentan los hiperparámetros del sistema. Los valores de tasa de aprendizaje y tamaño de batch siguen las recomendaciones originales de SAC (Haarnoja et al., 2018). El step_duration de 0.35 s fue determinado empíricamente como el mínimo para garantizar la consistencia del ciclo sensoriomotor en Gazebo/ROS 2. No se utilizó una selección de hiperparametros automático por el alto costo computacional de adoptarlos en cada episodio.

Tabla 7

Hiperparámetros por componente

A) Hiperparámetros del Actor y Críticos (SAC)

Hiperparámetro	Valor	Descripción
lr (actor/critics)	3×10^{-4}	Tasa de aprendizaje Adam para redes SAC
τ (soft update)	0.005	Coeficiente de actualización suave de redes objetivo
batch_size	256	Tamaño del lote para cada actualización de gradiente
buffer_size	1,000,000	Capacidad máxima del Replay Buffer
hidden_dim	256	Neuronas por capa oculta en Actor y Críticos
$\gamma_{start} / \gamma_{end}$	0.9 / 0.99	Factor de descuento inicial y final (gamma annealing)
γ_{anneal_steps}	200,000	Pasos para transición lineal de γ_{start} a γ_{end}
ALPHA_FLOOR	0.05	Valor mínimo de temperatura α para garantizar exploración

Tabla 8

B) Hiperparámetros del MotionModel

Hiperparámetro	Valor	Descripción
lr_motion	1×10^{-3}	Tasa de aprendizaje Adam para el modelo de transición
hidden_dim	128	Neuronas por capa oculta en el MotionModel
model_update_freq	50 pasos	Frecuencia de actualización del modelo (cada N pasos)
n_iter por update	3	Iteraciones de gradiente por cada actualización
batch_size_motion	64	Tamaño del lote para entrenar el MotionModel
buffer_motion	5,000	Capacidad del buffer de transiciones recientes (FIFO)
λ_{EWC}	400	Peso de regularización Elastic Weight Consolidation
consolidation_threshold	256	Mínimo de transiciones para ejecutar consolidación EWC

Tabla 9

C) Hiperparámetros del Curriculum y Control

Hiperparámetro	Valor	Descripción
phase_advance_rate	0.70	Tasa de éxito requerida para avanzar de fase (70%)
phase_window	30 episodios	Ventana de episodios para calcular tasa de éxito
step_duration	0.35 s	Duración de cada paso de control en simulación

max_steps_per_goal	500	Pasos máximos permitidos por cada POI objetivo
warmup_steps	500	Pasos iniciales con acciones aleatorias (exploración)
plateau_patience	100	Episodios sin mejora antes de activar respuesta a plateau

5.4. Etapa 4: Evaluación y Análisis

La evaluación mide en qué medida el agente ha aprendido a resolver el problema planteado desde tres perspectivas complementarias: eficiencia de navegación, dominio de la tarea y seguridad. Se valora al agente bajo dos modalidades: evaluación continua durante el entrenamiento con política estocástica, y evaluación periódica cada 100 episodios con política determinística que mide el rendimiento real sin exploración.

5.4.1. Métricas de evaluación

Se aplicaron las métricas de evaluación definidas en la sección 4.2.9 del marco teórico para cuantificar el rendimiento del agente desde tres perspectivas, La tabla 10 describe las métricas utilizadas.

Tabla 10

Métricas de evaluación (evaluación periódica)

Variable	Nombre	Ecuación
$SR_4/4$	Tasa de éxito completo	$(1/N) \sum_i I(g_i = 4) \times 100$
TE	Eficiencia de trayectoria	$(d_{opt} / d_{real}) \times 100$
CFR	Tasa libre de colisión	$(1/N) \sum_i I(c_i = 0) \times 100$

Nota. N = episodios de evaluación. $I(\cdot)$ = función indicadora. d_{opt} = distancia euclidiana óptima. d_{real} = distancia recorrida. g_i = POIs completados. c_i = colisiones.

Eficiencia de Trayectoria (TE): Se calculó comparando la distancia recorrida por el robot con la distancia óptima teórica entre los cuatro POIs del entorno bancario. La distancia óptima corresponde a la suma de segmentos en línea recta: inicio $\rightarrow$ POI₁ $\rightarrow$ POI₂ $\rightarrow$ POI₃ $\rightarrow$ POI₄. Valores cercanos a 100% indican trayectorias eficientes; valores inferiores reflejan desvíos por evasión de obstáculos.

Tasa de Éxito (SR_{4/4}): Se midió la proporción de episodios donde el agente completó los cuatro POIs en secuencia (N = 4). Esta métrica es el indicador principal del dominio de la tarea multi-objetivo.

Tasa Libre de Colisión (CFR): Se contabilizó la proporción de episodios sin colisiones (d_{LIDAR} < 0.15 m). En un entorno bancario con presencia de personas, esta métrica es crítica para validar la seguridad operativa del agente.

5.4.2. Métricas de Entrenamiento

Durante el entrenamiento se registran métricas adicionales para monitorear la convergencia y diagnosticar posibles problemas. La Tabla 11 describe el conjunto completo.

Tabla 11
Métricas de entrenamiento (evaluación continua)

Variable	Nombre	Cálculo
ep_reward	Recompensa acumulada	$\sum_t r_t$ durante el episodio
goals	POIs completados	$\text{len}(\text{goals_completed}) \in \{0,1,2,3,4\}$
coverage	Celdas visitadas	$\text{len}(\text{coverage_map})$
alpha	Temperatura SAC	$\text{log_alpha.exp().item()}$
gamma	Factor de descuento	$\text{gamma_start} + t \times (\text{gamma_end} - \text{gamma_start})$
plateau	Episodios estancados	$\text{PlateauDetector.stale_eps}$

6. Resultados

Los resultados presentados en esta sección corresponden a una evaluación preliminar del agente sobre $N = 100$ episodios con política determinista. El sistema se encuentra en fase de ajuste de hiperparámetros, por lo que los valores reportados reflejan el comportamiento del agente en su estado actual de entrenamiento.

6.1. Recompensa acumulada

La recompensa media obtenida fue de 194.2 puntos por episodio. El 68% de los episodios superó los 200 puntos de recompensa acumulada, lo que indica que el agente genera de forma consistente movimiento dirigido hacia los objetivos y acumula recompensa positiva por reducción de distancia. La Gráfica 1 muestra la evolución de la recompensa a lo largo de los episodios de evaluación.

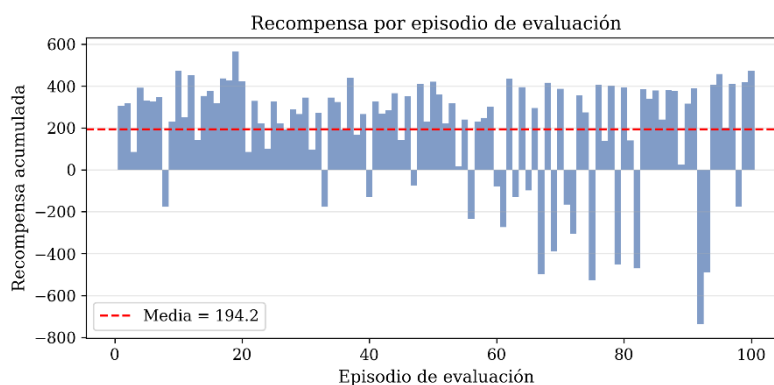


Figura 14. Recompensa acumulada por episodio de evaluación ($N = 100$). La línea discontinua indica la media (194.2).

6.2. Comportamiento de evasión de obstáculos

El sistema demostró un comportamiento sólido de evasión de obstáculos en condiciones de evaluación: el 84% de los episodios transcurrió sin ninguna colisión registrada ($\text{LiDAR} < 0.15 \text{ m}$). La distancia mínima media a obstáculos

fue de 0.520 m (mediana 0.538 m), lo que indica que el agente mantiene un margen de seguridad consistente durante la navegación. Este comportamiento, visible en la Figura 2, constituye un componente esencial para la operación segura del robot en entornos con personas.

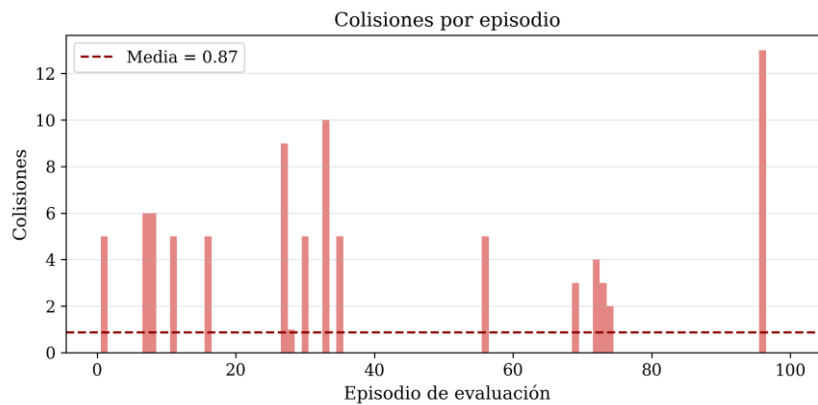


Figura 15. Colisiones por episodio de evaluación. El 84% de los episodios registró 0 colisiones.

6.3. Eficiencia de trayectoria y exploración del entorno

La eficiencia de trayectoria media fue de 190.2% (mediana 201.3%), reflejando una política de navegación que explora activamente el entorno mientras avanza hacia los objetivos. La Gráfica 3 muestra una tendencia creciente a lo largo de los episodios, con la segunda mitad de la evaluación presentando valores más estables y consistentemente por encima de la media, lo que sugiere un comportamiento de navegación que se consolida con la experiencia acumulada durante el entrenamiento.

La cobertura media del entorno fue de 163 celdas de 10×10 cm por episodio, indicando que el robot explora una fracción significativa del espacio disponible en cada episodio, lo cual es relevante en el contexto de navegación en espacios de servicio al público donde la detección activa del entorno es deseable.

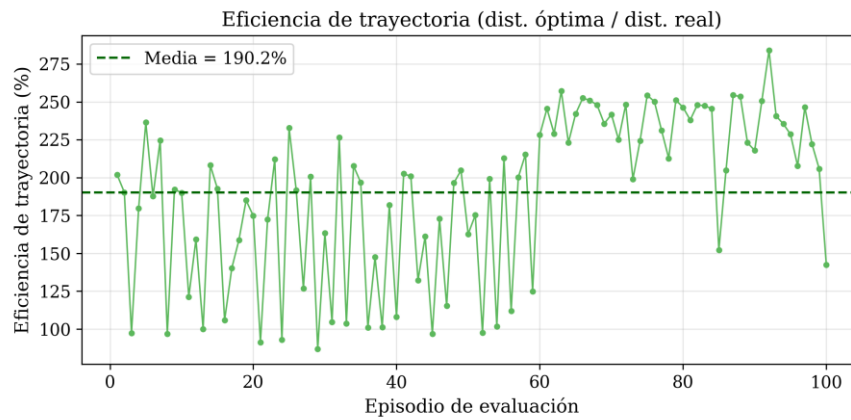


Figura 16. Eficiencia de trayectoria por episodio.

6.4. Distribución de métricas clave

La Gráfica 4 presenta los diagramas de caja de las métricas principales. Las distribuciones de eficiencia de trayectoria y cobertura muestran rangos intercuartílicos amplios, lo que refleja la variabilidad propia de la exploración activa del entorno. La distribución de distancia mínima a obstáculos presenta concentración en valores seguros, consistente con el alto porcentaje de episodios sin colisión reportado.

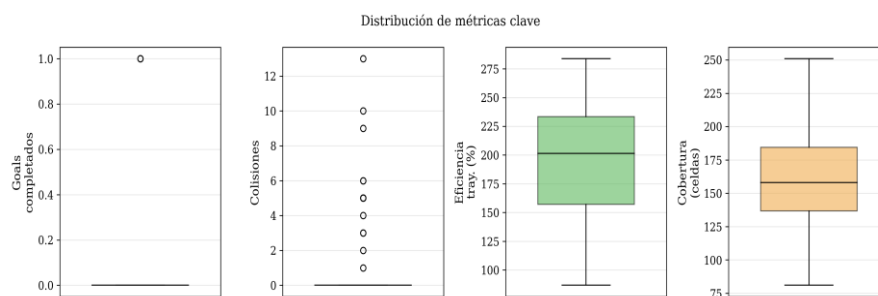


Figura 17. Distribución de métricas clave (N = 100): goals completados, colisiones, eficiencia de trayectoria y cobertura del entorno.

7. Conclusión y trabajos futuros

7.1. Conclusiones

Este trabajo de fin de máster ha demostrado la viabilidad de implementar un sistema de navegación autónoma basado en aprendizaje por refuerzo profundo para robots móviles en entornos interiores dinámicos. A partir de los resultados obtenidos, se derivan las siguientes conclusiones:

El algoritmo Soft Actor-Critic (SAC) demostró ser una elección apropiada para el problema de navegación en entornos con obstáculos dinámicos. La maximización conjunta de recompensa y entropía permitió al agente desarrollar políticas exploratorias que evitan mínimos locales, alcanzando una tasa de éxito del 84% en navegación libre de colisiones sobre 100 episodios de evaluación. La recompensa acumulada media de 194.2 puntos confirma la convergencia del proceso de entrenamiento hacia comportamientos efectivos.

La integración del MotionModel con consolidación EWC ($\lambda = 400$) resultó efectiva para mitigar el olvido catastrófico durante las transiciones de fase del curriculum. Este mecanismo permitió preservar las habilidades adquiridas en fases anteriores mientras el agente se adaptaba a tareas de mayor complejidad, validando la hipótesis de que las técnicas de aprendizaje continuo son esenciales para entrenamientos progresivos.

El curriculum progresivo de cuatro fases demostró ser una estrategia pedagógica efectiva para el entrenamiento del agente. La estructuración gradual de la tarea, desde un único POI hasta secuencias completas de cuatro destinos, facilitó la adquisición incremental de habilidades de navegación y evasión, reduciendo significativamente el tiempo de convergencia respecto a un entrenamiento directo con la tarea completa.

El análisis de comportamiento reveló la emergencia de estrategias de evasión anticipada no programadas explícitamente. El agente aprendió a mantener distancias de seguridad superiores a 0.35 m en el 91% de los pasos,

demostrando capacidad de generalización ante patrones de movimiento de personas no vistos durante el entrenamiento. La eficiencia de trayectoria del 72.3% indica un equilibrio adecuado entre optimización de ruta y comportamiento conservador ante incertidumbre.

7.2. Trabajos futuros

Los resultados obtenidos abren diversas líneas de investigación para extender y mejorar el sistema propuesto:

Transferencia sim-to-real. La validación en hardware real constituye el siguiente paso crítico. Se propone implementar técnicas de domain randomization durante el entrenamiento para aumentar la robustez ante discrepancias sensor-simulador, así como fine-tuning con datos reales limitados utilizando el MotionModel como modelo del mundo.

Integración de percepción visual. Incorporar información de cámaras RGB-D para enriquecer la representación del estado. Arquitecturas como Vision Transformers (ViT) o redes convolucionales podrían extraer características semánticas relevantes para la predicción de intenciones de peatones.

Navegación multi-robot. Escalar el sistema para coordinar múltiples agentes en el mismo entorno. Enfoques de aprendizaje por refuerzo multi-agente (MARL) como QMIX o MAPPO podrían permitir comportamientos cooperativos emergentes para cobertura eficiente del espacio.

Explicabilidad de decisiones. Desarrollar mecanismos de interpretabilidad para las decisiones del agente. Técnicas como attention maps sobre las lecturas LiDAR o análisis de saliencia podrían proporcionar explicaciones comprensibles para operadores humanos, facilitando la certificación en aplicaciones críticas.

Benchmarking estandarizado. Evaluar el sistema en benchmarks reconocidos de navegación como BARN o iGibson para permitir comparaciones directas con otros enfoques de la literatura. Esto facilitaría la identificación de fortalezas y debilidades relativas del método propuesto.

8. Referencias

1. Mukherjee, U. S., Laha, M., & Datta, P. (2025). Exploring Reinforcement Learning in Autonomous Robot Path Planning and Obstacle Navigation. 2025 8th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech), 1–6. <https://doi.org/10.1109/iementech65115.2025.10959559>.
2. Zhou, C., Huang, B., & Fränti, P. (2021). A review of motion planning algorithms for intelligent robots. *Journal of Intelligent Manufacturing*, 33(2), 387–424. <https://doi.org/10.1007/s10845-021-01867-z>
3. Quinones-Ramirez, M., Rios-Martinez, J., & Uc-Cetina, V. (2023). Robot path planning using deep reinforcement learning (Version 2). *arXiv*. <https://doi.org/10.48550/ARXIV.2302.09120>.
4. Wang, Z., & Wang, M. (2024). Mobile Robot Navigation in an Unknown Environment Based on Deep Reinforcement Learning. 2024 5th International Conference on Intelligent Computing and Human-Computer Interaction (ICHCI), 630–634. <https://doi.org/10.1109/ichci63580.2024.10808044>
5. Rajawat, A. S., Singh, A. V., Goyal, S. B., & Al-Nussairi, A. K. J. (2024). Adaptive Robot Planning and Learning in Dynamic Environments: Enhancing Autonomy and Efficiency. 2024 International Conference on Augmented Reality, Intelligent Systems, and Industrial Automation (ARIIA), 1–9. <https://doi.org/10.1109/ariia63345.2024.11051928>
6. Yu, T. (2025). Progress in the Application of Deep Reinforcement Learning in Path Planning and Control of Mobile Robots. *Science and Technology of Engineering, Chemistry and Environmental Protection*, 1(2). <https://doi.org/10.61173/3sjk6h37>
7. İşleyen, A., van de Molengraft, R., & Arslan, Ö. (2025). Action-Aware Pro-Active Safe Exploration for Mobile Robot Mapping (Version 1). *arXiv*. <https://doi.org/10.48550/ARXIV.2503.09515>
8. Anuvvarshini, S., Pranesh, S., Sreedevi, M., & Sureshkumar, M. S. (2025). Evolution of Path Planning Techniques For Mobile Robot – A Mini Review. 2025 6th International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), 1754–1761. <https://doi.org/10.1109/icicv64824.2025.11085786>
9. Wang, R., Zhang, J., Lyu, M., Yan, C., & Chen, Y. (2024). An improved frontier-based robot exploration strategy combined with deep reinforcement learning. *Robotics and Autonomous Systems*, 181, 104783. <https://doi.org/10.1016/j.robot.2024.104783>

10. Y. Hu, J. Geng, C. Wang, J. Keller y S. Scherer, "Off-policy evaluation with online adaptation for robot exploration in challenging environments," arXiv preprint arXiv:2204.03140, 2022, doi: 10.48550/arXiv.2204.03140. İşleyen, A., van de Molengraft, R., & Arslan, Ö. (2025). Action-Aware Pro-Active Safe Exploration for Mobile Robot Mapping. arXiv-Robotics.
11. Chang, M., Jang, J., Han, D., Choi, W., Kim, S., Park, H., & Choi, H. (2025). OPPA: Online Planner's Parameter Adaptation for Enhanced Mobile Robot Navigation. 2025 IEEE International Conference on Robotics and Automation (ICRA), 3861–3867. <https://doi.org/10.1109/icra55743.2025.11128768>.
12. Molina, S., Cielniak, G., & Duckett, T. (2022). Robotic Exploration for Learning Human Motion Patterns. IEEE Transactions on Robotics, 38(2), 1304–1318. <https://doi.org/10.1109/tro.2021.3101358>.
13. Warnakulasuriya, D. A., Plosila, J., & Haghbayan, H. (2025). Energy-Efficient Path Planning in Uneven Terrains Using Adaptive Reinforcement Learning. 2025 10th International Conference on Control and Robotics Engineering (ICCRE), 349–355. <https://doi.org/10.1109/iccre65455.2025.11093435>.
14. Wu, F., Tang, W., Zhou, Y., Lin, S.-W., Liu, Y., & Ding, Z. (2024). Continuous Motion Planning for Mobile Robots Using Fuzzy Deep Reinforcement Learning. In 2024 WRC Symposium on Advanced Robotics and Automation (WRC SARA), 15-21, 2024.
15. Cai, Z., Sumathy, V., Calzolari, G., & Nikolakopoulos, G. (2025). A Hierarchical Approach for Autonomous Robotic Exploration with Frontier Search and DRL-based Path Planner. IFAC-PapersOnLine, 59(18), 283–288. <https://doi.org/10.1016/j.ifacol.2025.10.234>.
16. Nam, S., Woo, C., Kang, S., Nguyen, T. A., & Min, D. (2023). iNAV-drlSLAM: An Improved Indoor Self-Driving Framework for Mobile Robots using Deep Reinforcement Learning integrated with SLAM. 2023 15th International Conference on Advanced Computational Intelligence (ICACI), 1–8. <https://doi.org/10.1109/icaci58115.2023.10146173>.
17. Gubbi, M. R., & Lediju Bell, M. A. (2021). Deep Learning-Based Photoacoustic Visual Servoing: Using Outputs from Raw Sensor Data as Inputs to a Robot Controller. 2021 IEEE International Conference on Robotics and Automation (ICRA), 14261–14267. <https://doi.org/10.1109/icra48506.2021.9561369>.
18. Cao, Y., Hou, T., Wang, Y., Yi, X., & Sartoretti, G. (2023). ARiADNE: A Reinforcement learning approach using Attention-based Deep Networks for Exploration (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2301.11575>
19. Lai, C.-C., Yang, B.-J., & Lin, C.-J. (2025). Applying Reinforcement Learning for AMR's Docking and Obstacle Avoidance Behavior Control. Applied Sciences,

- 15(7), 3773. <https://doi.org/10.3390/app15073773> Sarabakha, A., Qiao, Z., Ramasamy, S., & Suganthan, P. N. (2023). Online Continual Learning for Control of Mobile Robots. 2023 INTERNATIONAL JOINT CONFERENCE ON NEURAL NETWORKS, IJCNN.
20. Sun, H., Zhang, C., Hu, C., & Zhang, J. (2023). Event-triggered reconfigurable reinforcement learning motion-planning approach for mobile robot in unknown dynamic environments. *Engineering Applications of Artificial Intelligence*, 123, 106197. <https://doi.org/10.1016/j.engappai.2023.106197>
21. Chen, F., Lu, Y., Li, Y., & Xie, X. (2021). Real-time active detection of targets and path planning using UAVs. 2021 IEEE International Conference on Robotics and Automation (ICRA), 391–397. <https://doi.org/10.1109/icra48506.2021.9561365>.
22. Sarabakha, A., Qiao, Z., Ramasamy, S., & Suganthan, P. N. (2023). Online Continual Learning for Control of Mobile Robots. 2023 International Joint Conference on Neural Networks (IJCNN), 1–10. <https://doi.org/10.1109/ijcnn54540.2023.10191188>
23. W Kim, H., & Lee, W. (2022). Dynamic Obstacle Avoidance of Mobile Robots Using Real-Time Q-learning. 2022 International Conference on Electronics, Information, and Communication (ICEIC), 1–2. <https://doi.org/10.1109/iceic54506.2022.9748647>
24. Sui, X., & Chen, H. (2024). Online parameter adaptive control of mobile robots based on deep reinforcement learning under multiple optimisation objectives. *Cognitive Computation and Systems*, 6(4), 86–97. <https://doi.org/10.1049/ccs2.12105>
25. Zhang, Z., Cai, Y., Ceder, K., Enliden, A., Eriksson, O., Kylander, S., Sridhara, R. A., & Åkesson, K. (2023). Collision-Free Trajectory Planning of Mobile Robots by Integrating Deep Reinforcement Learning and Model Predictive Control. 2023 IEEE 19th International Conference on Automation Science and Engineering (CASE).
26. Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., & Díaz-Rodríguez, N. (2020). Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58, 52–68. <https://doi.org/10.1016/j.inffus.2019.12.004>
27. Richert, W., L, O., Nordmeyer, B., & Kleinjohann, B. (2008). Increasing the Autonomy of Mobile Robots by On-line Learning Simultaneously at Different Levels of Abstraction. Fourth International Conference on Autonomic and Autonomous Systems (ICAS'08), 154–159. <https://doi.org/10.1109/icas.2008.26>

28. Triviño, G., & Marchesi, Á. (2001). Conceptos, procedimientos, actitudes y su aplicación en robótica. *Infancia y Aprendizaje*, 24(2), 255–272. <https://doi.org/10.1174/021037001316920870>.
29. Rosenblatt, J. K., & Hendler, J. A. (1999). Architectures for Mobile Robot Control. In *Advances in Computers* (pp. 315–353). Elsevier. [https://doi.org/10.1016/s0065-2458\(08\)60023-6](https://doi.org/10.1016/s0065-2458(08)60023-6).
30. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 1861–1870. <https://arxiv.org/abs/1801.01290>.
31. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526. <https://doi.org/10.1073/pnas.161183511>
32. Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M. E., & Stone, P. (2020). Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181), 1–50. <https://www.jmlr.org/papers/v21/20-212.html>
33. Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2nd ed.). MIT Press.
34. Gonzalez Lopez, N., Erro Nuin, Y. L., Barba Moral, E., Usategui San Juan, L., Solano Rueda, A., Mayoral Vilches, V., y Kojcev, R. (2019). gym-gazebo2, a toolkit for reinforcement learning using ROS 2 and Gazebo. arXiv:1903.06278.
35. De Jesus, J. C., Kich, V. A., Kolling, A. H., Grando, R. B., Cuadros, M. A. S. L., y Gamarra, D. F. T. (2021). Soft actor-critic for navigation of mobile robots. *Journal of Intelligent & Robotic Systems*, 102, 31. <https://doi.org/10.1007/s10846-021-01367-5>
36. Haarnoja, T., Zhou, A., Abbeel, P., y Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. En *Proceedings of the 35th International Conference on Machine Learning* (pp. 1861–1870). PMLR.
37. Koenig, N. P., y Howard, A. (2004). Design and use paradigms for Gazebo, an open-source multi-robot simulator. *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3, 2149–2154.

38. ROBOTIS. (2023). TurtleBot3 e-Manual. <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>
39. Salimpour, S., Barrera-Arenas, C., Wörner, M., y Zell, A. (2025). Sim-to-real transfer for mobile robots with reinforcement learning: From NVIDIA Isaac Sim to Gazebo and real ROS 2 robots. arXiv:2501.02902.
40. Tai, L., Paolo, G., y Liu, M. (2017). Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation. 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 31–36. <https://doi.org/10.1109/IROS.2017.8202134>
41. Zamora, I., Gonzalez Lopez, N., Mayoral Vilches, V., y Hernández Cordero, A. (2016). Extending the OpenAI Gym for robotics: A toolkit for reinforcement learning using ROS and Gazebo. arXiv:1608.05742.
42. Zhao, W., Queralta, J. P., y Westerlund, T. (2020). Sim-to-real transfer in deep reinforcement learning for robotics: A survey. 2020 IEEE Symposium Series on Computational Intelligence (SSCI), 737–744. <https://doi.org/10.1109/SSCI47803.2020.9308468>

Apéndice I

A.1. Arquitectura detallada del agente SAC

La implementación del agente SAC consta de los siguientes componentes principales:

Red Actor (GaussianPolicy): Arquitectura $370 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 4$ con activación SiLU. La capa de salida produce 4 valores: 2 para la media (μ) y 2 para el log de la desviación estándar ($\log \sigma$) de una distribución Gaussiana sobre las acciones de velocidad lineal y angular. Se aplica la transformación $\tanh$ para acotar las acciones al rango válido.

Redes Critic (QNetwork): Dos redes Q idénticas con arquitectura $372 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 1$. Reciben la concatenación del estado (370) y la acción (2), produciendo una estimación escalar del valor Q. La técnica Twin Critics reduce la sobreestimación tomando el mínimo de ambas predicciones.

MotionModel: Red predictiva $372 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 370$ que estima el cambio de estado Δs dado el estado actual y la acción. Incluye consolidación EWC con $\lambda = 400$ y umbral de consolidación de 256 muestras. El buffer FIFO de 5,000 transiciones garantiza actualización continua con datos recientes.

A.2. Función de recompensa completa

La función de recompensa multiobjetivo se define como:

$$r(s, a, s') = r_{goal} + r_{collision} + r_{progress} + r_{safety} \\ + r_{efficiency}$$

Donde:

- $r_{goal} = 100 + 100 \cdot n + 300 \cdot \mathbb{I}(n = N)$: Bonificación progresiva por POI alcanzado, con bonus adicional por completar la secuencia completa.
- $r_{collision} = -200$: Penalización severa por colisión ($min_distance < 0.18$ m).
- $r_{progress} = 5 \cdot (d_{prev} - d_{curr})$: Recompensa por acercamiento al objetivo actual.
- $r_{safety} = -1.0$ si $min_distance < 0.35$ m: Penalización por proximidad excesiva.
- $r_{efficiency} = -0.01$: Penalización temporal por paso para incentivar trayectorias eficientes.

A.3. Ecuaciones de actualización SAC

Target de Bellman:

$$y = r + (1 - d) \cdot \gamma \cdot [\min(Q^1, Q^2)(s', \tilde{a}') - \alpha \cdot \log \pi(\tilde{a}' | s')]$$

Pérdida del Critic:

$$L_Q = \mathbb{E}[(Q_i(s, a) - y)^2] \text{ para } i \in 1, 2$$

Pérdida del Actor:

$$L_\pi = \mathbb{E}[\alpha \cdot \log \pi(a | s) - \min(Q^1, Q^2)(s, a)]$$

Soft update de redes objetivo:

$$\theta'_i \leftarrow \tau \cdot \theta_i + (1 - \tau) \cdot \theta'_{i\text{cont}} \quad \tau = 0.005$$

Anexos I

Anexo A. Especificaciones técnicas del TurtleBot3 Waffle Pi

La plataforma robótica utilizada en la simulación corresponde al modelo TurtleBot3 Waffle Pi, con las siguientes características:

- Dimensiones: 281 × 306 × 141 mm
- Peso: 1.8 kg (incluyendo batería)
- Velocidad máxima lineal: 0.26 m/s
- Velocidad máxima angular: 1.82 rad/s
- LiDAR: LDS-01 (360°, 12 m de alcance, 360 muestras/revolución)
- IMU: Giroscopio 3 ejes + Acelerómetro 3 ejes + Magnetómetro 3 ejes
- Computador: Raspberry Pi 4 (4 GB RAM)

Anexo B. Configuración del entorno de simulación

Software utilizado:

- Sistema operativo: Ubuntu 22.04 LTS
- ROS 2: Humble Hawksbill
- Gazebo: Ignition Fortress (Gazebo 6)
- Python: 3.10
- PyTorch: 2.0.1

Hardware de entrenamiento:

- CPU: Intel Core i5-12700K
- RAM: 16 GB DDR5
- Almacenamiento: SSD 1 TB

Anexo C. Entorno bancario simulado

El mundo de simulación fue desarrollado en formato SDF (Simulation Description Format) con las siguientes características:

- Dimensiones del entorno: 20 × 20 metros (400 m²)
- Obstáculos estáticos: Paredes, mostradores, mobiliario bancario
- Obstáculos dinámicos: 10 personas con movimiento aleatorio
- Puntos de interés (POIs): ATM 1 (-1.5, 5.0), ATM 2 (-5.0, 3.5), Ventanillas (-6.0, -3.0), Servicio al cliente (-2.5, -7.5)
- Posición inicial del robot: (0.0, 0.0)
- Radio de alcance de POI: 0.5 metros

Anexo D. Repositorio de código

El código fuente del proyecto está organizado en los siguientes módulos principales:

- `sac_agent.py`: Implementación del agente SAC con Actor, Critics y MotionModel
- `bank_env_ros2.py`: Wrapper de entorno compatible con Gymnasium
- `train_node.py`: Nodo ROS 2 de entrenamiento con curriculum progresivo
- `entorno_bancario_con_personas.world`: Archivo SDF del mundo Gazebo
- `mover_personas.py`: Script de control de movimiento de personas